EquaCert: Verifying and Extending Completeness Results for Quantum Circuits in Agda

ANONYMOUS AUTHOR(S)

A set of circuit equations is complete if every equation between circuits with the same semantics follows from it, and minimal if no equation can be derived from the others. Proofs of these properties take long case analyses spanning hundreds of pages, making the details hard to check by hand. To address this, we extend an existing Agda library into EquaCert, a framework that divides the work between humans and machines. Humans set up a unified framework of circuits, matrix semantics, and group presentations, and state each paper's main theorem using these definitions. LLM coding agents then write the proofs following the papers' arguments, and the proof assistants check them. A human therefore need only review each paper's relations, gate semantics, and theorem statement. Using EquaCert, we check sixteen published results, fifteen on completeness and one on minimality. They cover discrete gate sets such as Clifford, Clifford+T, and Toffoli–Hadamard, on qubits or qudits of every odd prime dimension. We verify fourteen of them and repair errors in each of the other two. Reusing EquaCert's verified libraries, we also prove a new result: seven equations are complete for four-bit reversible circuits over {X, CX, CCX}. Under human supervison, the agents translated EquaCert into Lean, making it available to a broader mathematical community. We expect EquaCert to help with other open completeness problems, especially for quantum circuits in higher dimensions.

1 Introduction

Rewriting of quantum circuits is a ubiquitous tool in circuit optimization [44, 54, 55, 70], equivalence checking [71], hardware mapping [68], and reasoning about quantum programs [32]. This relies on an *equational theory* for a family of quantum circuits, a set of rewrite rules that transform a circuit into an equivalent one. Such a theory should be *sound* so that every rewrite preserves the underlying linear map. It should also be *complete* so that any equivalence between circuits can be derived. A theory is *minimal* if no rule can be derived from the others. Minimality is mainly of mathematical interest, because a minimal theory reveals the essential algebraic structure of a circuit family [19, 26].

Completeness results have been established for a wide range of circuit families, including reversible linear circuits [47, 63], reversible circuits over multiple-control Toffoli gates [33, 34], Clifford [12, 48, 53, 65], CNOT-dihedral [1, 18], two-qubit Clifford+T [15], three-qubit Clifford+CS [16], Toffoli–Hadamard [3], and real Clifford+CH [25] circuits, as well as universal quantum circuits [27, 28]. Completeness has also been obtained for the group of two-level matrices underlying several of these circuit families [14, 24, 32, 38, 49]. Minimality has been shown for six of these families [19] and for universal quantum circuits [26]. Proofs of both properties typically follow a simple strategy, but carrying it out requires long case analyses, with supporting derivations spanning hundreds of pages [25, 32, 48].

Peer review cannot check proofs of this volume in detail. When a derivation takes many rewrite steps, one step can easily rely, through earlier lemmas, on the very equation being derived. A *proof assistant* such as Lean or Agda can check these dependencies and reject such circular arguments. Formalizing these proofs would not have been feasible in the past, given the sheer number of lemmas and lengthy rewrites in these papers. Recent LLMs can write correct code for proof assistants, which brings this closer to reality. For example, Anthropic reports that LLMs wrote a 13-million-line Lean proof of Fermat's Last Theorem within 11 days [8].

In quantum computing, however, proof assistants and LLM agents have mostly been used to verify programs rather than circuit equational theories. Verified compilers and program logics prove that quantum programs are correct, or that circuit optimizations preserve their meaning [22, 44, 60, 74]. AI-assisted work has also formalized quantum program logics, algorithms, and complexity results [50, 62, 73].

Only a few completeness results for circuits have been formalized, each on its own and mostly by the authors of the result. Choudhury et al. [23] prove in Agda that the equations of the reversible programming language Π are complete for permutations of finite sets. Bian and Selinger formalized their completeness proofs for two-qubit Clifford+T and three-qubit Clifford+CS circuits [15, 16].

This isolation means that each formalization must trust the results it builds on. For example, Clément's completeness result for real Clifford+CH circuits [25] relies on the presentation of $O_n(\mathbb{Z}[1/\sqrt{2}])$ in Hadamard-Π [32], whose proof has a gap. Similarly, the quantum extension of Π is complete for Clifford and related gate sets [21] because it derives the presentations of [14, 15, 65]. Formalizing many results in one framework makes such dependencies checkable, lets reviewers audit the shared definitions once, and lets each new formalization reuse earlier ones.

Our Contributions. We present EquaCert, a framework in Agda [20, 57] and Lean [30] for verifying the completeness and minimality of circuit equational theories. Figure 1 summarizes its workflow. First, humans state each paper's main theorem and check that it, together with the definitions and relations it uses, faithfully expresses the source paper. Then, LLM coding agents formalize the proof. Finally, Lean or Agda checks it.

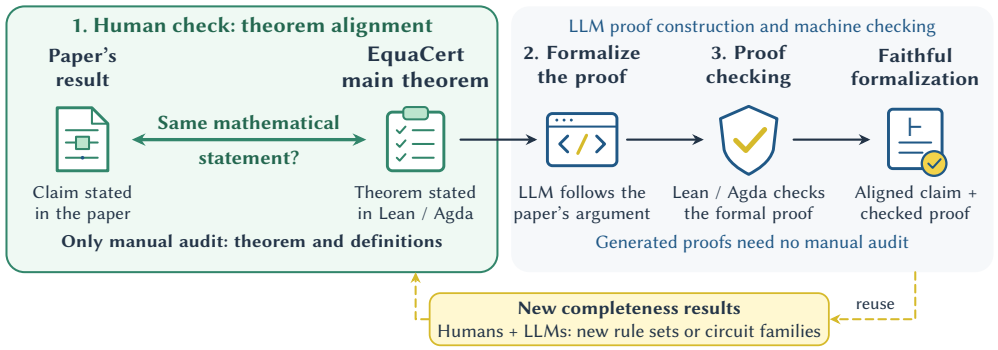


Fig. 1. EquaCert's workflow. The main path checks published results; the dashed branch reuses the verified libraries for human-guided, LLM-assisted work on new completeness results. Proposed results undergo the same human review of definitions and theorem statements, and the same machine checking of formal proofs.

The manual audit matters because a proof assistant checks a proof only against the theorem as stated, which may not match the intended result [42]. For example, Bastounis et al. [9] find that, because of mistranslations, a claimed Navier–Stokes formalization verified a weaker statement than intended. EquaCert therefore states each paper's main theorem in its own module, in both Agda and Lean. Since the proof assistant checks every proof, a reviewer need only compare these statements, and the definitions they use, with the source papers to confirm that a formalization is faithful.

EquaCert builds on two earlier Agda developments. The first is the formalization [13] of the multi-qudit Clifford completeness result of Bian, Li et al. [12]. The second is the library of circuits and group presentations of Bian and Feng [11], which EquaCert generalizes by adding matrix semantics, minimality, three-wire gates, and scalar generators. Once the Agda framework was complete, we instructed LLM coding agents to translate it into Lean, which has a large community that uses it to formalize mathematics [58, 69], build AI theorem provers [72], and formalize quantum information theory and quantum programs [10, 45, 67, 75]. As a result, EquaCert can build on Lean's well-developed mathematical library Mathlib. In the long run, the Lean community can use

EquaCert's verified rule sets and completeness theorems to justify circuit rewrites in a verified quantum program.

We then use EquaCert to check sixteen papers, which include most of the completeness results for quantum circuits over discrete gate sets. We verify fourteen of them and correct errors that peer review had missed in the other two. Most of these circuit families are restricted Clifford+T circuits. Amy et al. [2] show that several of these circuit families, with at most one ancilla, represent exactly the unitary matrices over a subring of $\mathbb{Z}[1/\sqrt{2}, i]$. Figure 2 displays these gate sets, their matrix groups, and multi-level generators. Because these families are nested and their matrices all have entries in $\mathbb{Z}[1/\sqrt{2}, i]$, EquaCert reuses exact arithmetic, matrix semantics, group constructions, and completeness arguments across formalizations.

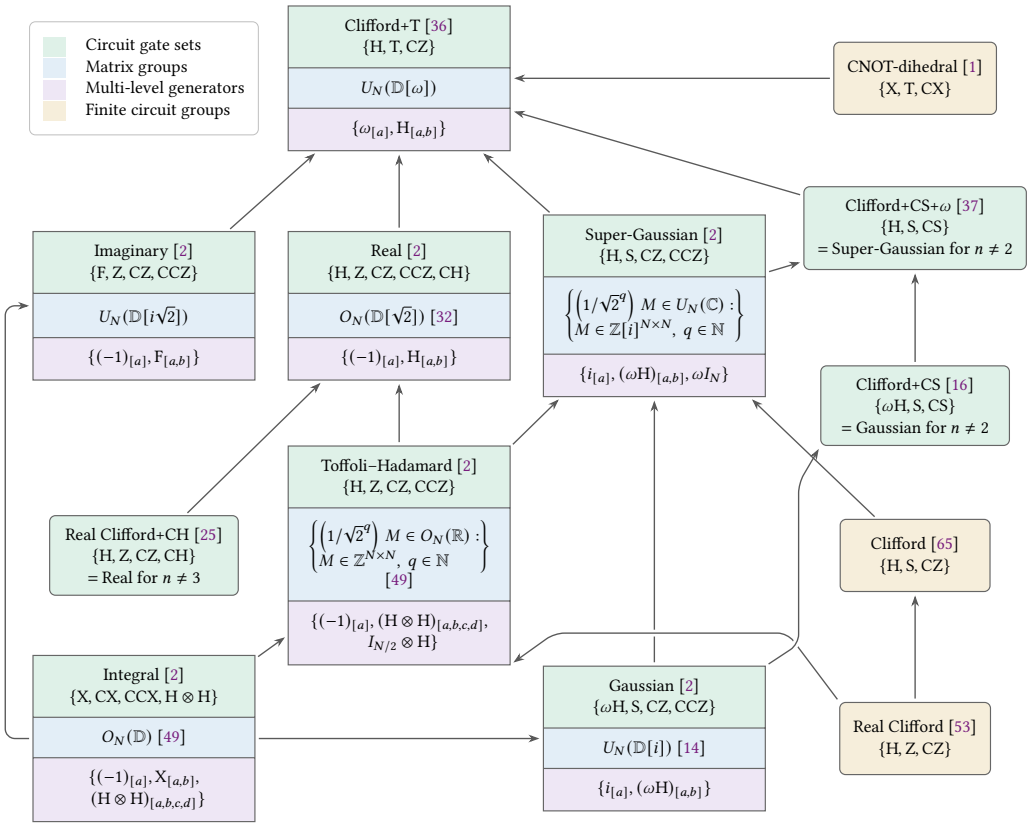


Fig. 2. Restricted Clifford+T circuit families and their underlying matrix groups, with $N = 2^n$ and $n \geq 1$. Each matrix group is the one its circuits represent with at most one clean ancilla. Arrows denote inclusion of ancilla-free circuit groups and, where shown, matrix groups. Mint-only boxes mark when related circuit groups agree; each exceptional inclusion is strict. Here $\mathbb{D} = \mathbb{Z}[1/2]$, $\omega = e^{i\pi/4}$, $\mathrm{F} = (\mathrm{H} + iI)/\sqrt{2}$, and q ranges over nonnegative integers. Subscripts specify distinct basis levels; $I_{N/2} \otimes \mathrm{H}$ and ωI_N act globally. In both Gaussian cases, $\mathrm{X}_{[a,b]} = (\omega\mathrm{H})_{[a,b]} i^2_{[b]} (\omega\mathrm{H})^\dagger_{[a,b]}$ is redundant. Clifford+CS+ω denotes adjoining the scalar ω to Clifford+CS.

Beyond verifying published results, EquaCert facilitates the discovery of new ones by freeing researchers from labour-intensive checking so they can focus on developing new techniques. Until

now, Toffoli–Hadamard circuits had a complete equational theory only on three qubits [3], and its rule set omits a relation that EquaCert shows to be independent. Working with LLM coding agents, we combine the presentation of $O_n(\mathbb{Z}[1/2])$ by Li et al. [49] with the method of Clément [25] to extend this completeness to any number of qubits. Along the way, we obtain a complete equational theory for circuits over {X, CX, CCX}. EquaCert then checks and confirms both proofs, building on its existing formalizations of the presentation and the method they combine.

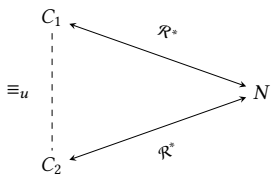
Concurrent Work. Several concurrent projects use a similar idea, where humans check the statements and LLM agents write the proofs. MerLean formalizes three quantum computing papers [61], and EconCSLib formalizes twenty papers in economics and computation [35]. An LLM agent has also proved part of a verified compiler correct [59]. QECert collects results of many papers on quantum error correction in one Lean library [43]. The closest to EquaCert is Shirobokov et al. [67], who apply this idea to quantum programs, with LLM agents writing both the programs and their proofs.

Orientation. Sections 2 and 3 introduce the proof strategies and the EquaCert framework, and Sections 4 and 5 present the formalized results and the corrections to the source papers. Section 6 gives a new result, a complete equational theory for four-bit reversible circuits over {X, CX, CCX}, proved in Appendix E. Section 7 concludes.

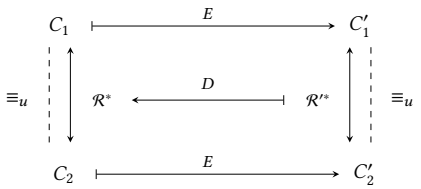
2 Preliminaries

A circuit is a word over an alphabet $\mathcal{G}$ of gates. A rewrite rule is then a relation between words over this alphabet, and a rule set $\mathcal{R}$ generates a congruence on circuits. To a circuit C we associate an *interpretation* $[\![C]\!]$, which is its underlying matrix. The completeness results we formalize have matrix entries in the finitely generated rings $\mathbb{Z}\left[1/\sqrt{2}, i\right]$, $\mathbb{Z}\left[1/2, i\right]$, $\mathbb{Z}\left[1/\sqrt{2}\right]$, and $\mathbb{Z}\left[1/2\right]$, which make them exactly representable in Agda. We leave the formalization of universal circuits taking values in $\mathbb{C}$ for future work.

There are two main strategies for proving completeness of a rule set $\mathcal{R}$ over a gate set $\mathcal{G}$, both of which we support. Consider any two circuits C_1 and C_2 over $\mathcal{G}$ that have the same linear map, $[\![C_1]\!] = [\![C_2]\!]$. We need to show that C_1 can be rewritten into C_2 using $\mathcal{R}$. Figure 3a outlines the first strategy. It specifies a unique normal form for each linear map [1, 12, 18, 33, 34, 47, 48, 53, 63, 65], often by giving an exact synthesis algorithm [14, 24, 32, 38, 49]. Completeness follows by proving that every circuit can be transformed into its normal form using the rewrite rules. By uniqueness, C_1 and C_2 have the same normal form N. Reversing the path from C_2 to N and concatenating it with the path from C_1 to N then gives the desired path of rewrites.



(a) The rewrites from C_1 and C_2 to their common normal form N using $\mathcal{R}$ give rewrites from C_1 to C_2 using $\mathcal{R}$.



(b) Completeness of $\mathcal{R}'$ gives rewrites between $C_1' = E(C_1)$ and $C_2' = E(C_2)$. Translating this proof back gives rewrites between C_1 and C_2 using $\mathcal{R}$.

Fig. 3. Two strategies for proving completeness of $\mathcal{R}$ over a gate set $\mathcal{G}$. $\overset{\mathcal{R}^*}{\longleftrightarrow}$ means applying a finite sequence of rewrites using $\mathcal{R}$ in either direction, where $*$ denotes zero or more steps.

Figure 3b illustrates the second strategy, which uses an already complete rule set $\mathcal{R}'$ over a different gate set $\mathcal{G}'$ [3, 15–17, 19, 25, 27, 28, 31, 64]. Here, E and D translate circuits from $\mathcal{G}$ to $\mathcal{G}'$ and back, replacing each gate with a circuit over the other gate set. Both E and D preserve circuit composition and the underlying linear map. To transfer completeness from $\mathcal{R}'$ to $\mathcal{R}$, we first show that every rule of $\mathcal{R}'$, translated by D, can be derived using $\mathcal{R}$. We can then replace each step of the proof between C_1' and C_2' by its translated derivation, giving $D(C_1') \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_2')$. It then remains to prove that the rewrite $C_j \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_j')$ is derivable, for which it suffices to prove the round-trip rewrites $g \overset{\mathcal{R}^*}{\longleftrightarrow} D(E(g))$ for each $g \in \mathcal{G}$. Sometimes the two strategies are combined, as in the scaled dyadic extension of [49].

The exact synthesis results used by the first strategy present groups of matrices rather than circuits (the first block of Table 1). Their generators are *multi-level* matrices. For an $m \times m$ matrix M and distinct indices $a_1, \ldots, a_m$ in $\{0, \ldots, n-1\}$, the *m-level* matrix $M_{[a_1,\ldots,a_m]}$ is the $n \times n$ matrix that acts as M on the standard basis vectors $e_{a_1}, \ldots, e_{a_m}$ and as the identity on the others. For example, $(-1)_{[a]}$ negates e_a, and $\mathrm{X}_{[a,b]}$ exchanges e_a and e_b. Circuits on k qubits have matrices of dimension $n = 2^k$, but these groups are defined in every dimension n. We introduce multi-level matrices for two reasons. The gap we find in Section 5 is in a presentation by one- and two-level matrices [32], and our Toffoli–Hadamard completeness result builds on a presentation by one-, two-, and four-level matrices [49].

For proofs based on exact synthesis, EquaCert supplies the common induction argument. Each development proves a Main Lemma connecting the algorithm's reduction steps to the rewrite rules. This isolates the algorithm-specific case analysis—the main work in each formalization—from the reusable completeness proof. Appendix A gives the argument.

Starting from a complete rule set, we can remove a redundant rule by deriving it from the other rules, thereby preserving completeness. To show that the resulting rule set is minimal, we need to give, for each remaining rule R, an interpretation of circuits that every other rule preserves but R does not. Any derivation using the other rules preserves this interpretation, so R cannot be derived from them [19, 26].

3 The EquaCert Framework

Our Agda framework EquaCert states every completeness theorem as a group presentation. Gates are indexed by their arity, the number of wires on which they act. Each source relation is stated on its original wires, and shared *structural rules* allow it to be used inside a larger circuit. These rules are independent of the gate set and make explicit the circuit structure that source papers often leave implicit.

We associate to circuits a matrix semantics $[\![-]\!]$ with entries in a finitely generated ring such as $\mathbb{Z}\left[1/\sqrt{2}, i\right]$. This defines a group homomorphism to a unitary matrix group G, whose group structure is checked once per ring using Agda's standard library. EquaCert extends the interpretation from gates to circuits and proves the structural rules sound once for all gate sets. For soundness, each development then needs to check only its own rules.

Soundness and completeness say the relations present the image of $[\![-]\!]$, which is the subgroup of G generated by the gates. In Agda, mathematical structures such as groups are defined as records. The record $\mathcal{R}$ IsSubPresentationOf G contains the semantics $[\![-]\!]$, with proofs of soundness and completeness. The record $\mathcal{R}$ IsPresentationOf G adds surjectivity, so the relations present all of G.

Figure 4 summarizes the two obligations: checking each rule's matrix equality for soundness, and deriving a known complete rule set $\mathcal{T}$ from $\mathcal{R}$ for completeness.

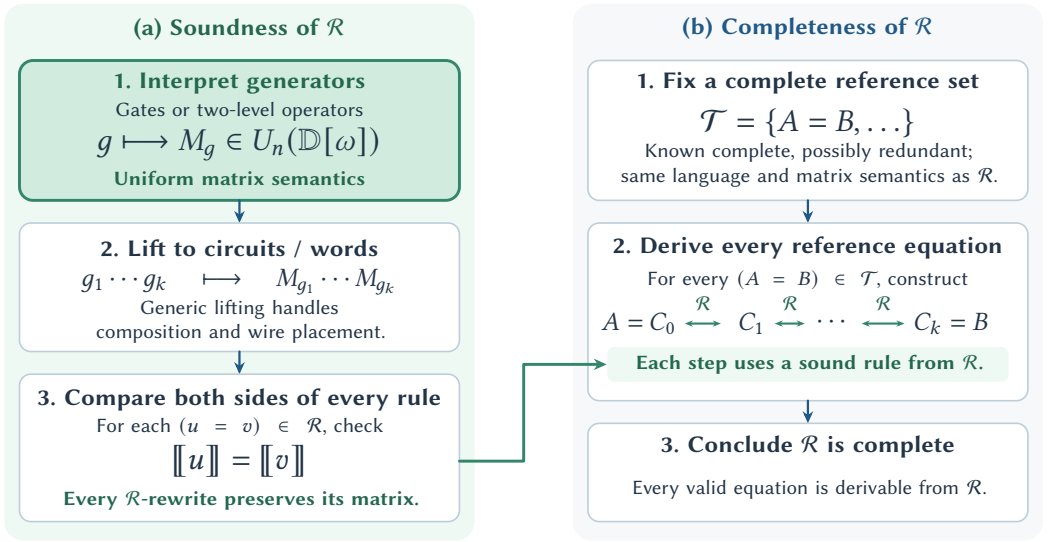


Fig. 4. EquaCert's shared matrix interpretation supports both proof obligations. The left column checks every rule of $\mathcal{R}$; the right derives every rule of a known complete set $\mathcal{T}$ in the same circuit language and matrix semantics. Each rewrite step uses a rule of $\mathcal{R}$, whose soundness is checked on the left. The displayed $U_n(\mathbb{D}[\omega])$ covers qubit fragments [2], with $\mathbb{D} = \mathbb{Z}[1/2]$ and $\omega = e^{i\pi/4}$. Qudit circuit families use other rings.

Each formalization ends with one of these records. For example, Statement (1) states that the real stabilizer relations [53] present the subgroup generated by their gates, while Statement (2) states that the Hadamard-Π relations [32] present the specified orthogonal group. The fields in Statements (3) and (4) state the two directions: equal matrices imply a derivable circuit equation, and every derivable equation preserves the matrix.

```
presentation : ∀ n → (n CRel,_===_) IsSubPresentationOf U (2 ^ n)          (1)
presentation : ∀ n → (n MRel,_===_) IsPresentationOf (O n)                 (2)
complete : ∀ {n} {w v : Circuit n} → [[ w ]] ≈ᵁ [[ v ]] → ⊢ w ≈ v          (3)
sound : ∀ {n} {w v : Circuit n} → ⊢ w ≈ v → [[ w ]] ≈ᵁ [[ v ]]             (4)
```

The minimality statement in Statement (5) says that the rule set is complete on n wires and becomes incomplete there if any rule is removed [19].

```
minimal-complete : ∀ {n} → (n ⊢_===_) IsMinimal × (n ⊢_===_) IsComplete     (5)
```

For each presentation, a reader checks four components against the source paper: the transcribed relations, the target group, the matrix of each gate, and the assignment of $[\![-]\!]$ to the record. Each Agda development collects these components and its main theorem in a MainTheorem module. The file `MainTheorems.agda` restates the results; the corresponding Lean statements are in each development's `Theorem` module, which `QCert/Table1.lean` imports.

Extensions of the Earlier Framework. Besides matrix semantics and minimality, EquaCert extends the one- and two-wire syntax of Bian and Feng [11] with three-wire gates such as Toffoli and explicit scalar generators for expressing exact equality, including global phase. It also supplies the structural swap laws of PROPs [51], so each paper need only state its own relations. Appendix B gives the encoding conventions, theorem notation, and implementation details.

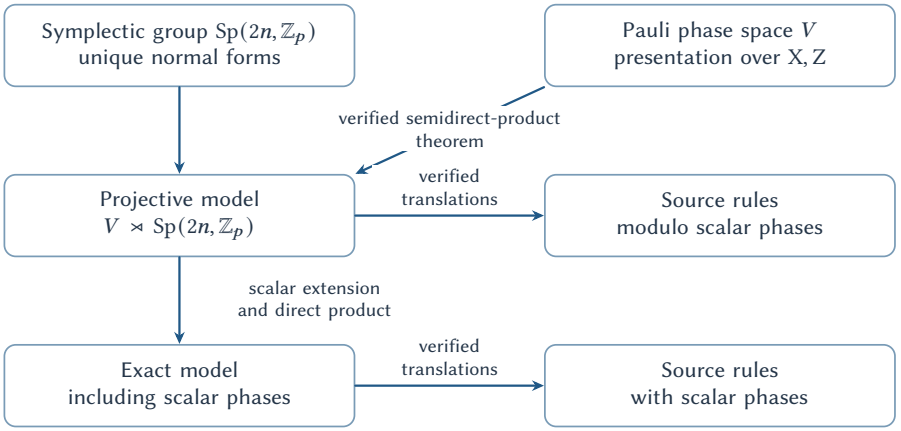


Fig. 5. The formalization of the multi-qudit Clifford result of Bian, Li et al. [12], assembled from verified components. The symplectic normal-form proof and the Pauli presentation combine into a projective model; restoring phases gives an exact model. Arrows denote checked constructions and translations of presentations. Appendix D gives the full case study, including the intermediate rule sets (Appendix D.6) and the scalar corrections (Appendix D.7).

3.1 Formalization Workflow

Our workflow has three parts. First, before formalizing a paper, we check its rewrite rules, its intermediate results, and each line of its derivations with exact matrices in short scripts. This caught most typos before any Agda code was written. Second, we state each main theorem in EquaCert before a coding agent starts, so the agent only supplies the proof. Finally, for long derivations, the agent writes a script that turns each line of the paper into one step of the Agda proof, so a failing step points to a specific line of the paper.

Direct evaluation of long matrix products was expensive, so EquaCert uses a shared exact procedure for matrix equality. We record this technique in the agents' instructions so that later formalizations reuse it; Appendix B.3 gives the implementation.

3.2 Avoiding Completeness Lost in Translation

For a formalization to be useful, its formal statement must say what the source paper claims. We make this connection explicit in three ways.

First, we define the target group independently of the circuit syntax, using matrices or an algebraic construction. A completeness theorem then relates the circuit equations to a semantics that does not depend on them, rather than to a group built from the equations themselves.

Second, when a source rule set must be changed, the formal theorem names the revised set. This lets a reader tell a repair to the proof apart from a correction to the theorem itself (Section 5).

Finally, we verify general completeness arguments and group constructions once, including Reidemeister–Schreier coset enumeration [52] and presentations of semidirect products. Each formalization that uses them only needs to prove their hypotheses and check the connection to its own circuit semantics.

For example, Figure 5 shows how we formalize the multi-qudit Clifford result of Bian, Li et al. [12] using the first and third choices. Its semantic model is the semidirect product of the Pauli phase space by the symplectic group, which we define independently of the circuits. To prove

completeness, we combine a normal-form proof for the symplectic group with the framework's semidirect-product and scalar-extension theorems. We give the full case study in Section D.

4 Formalized Completeness and Minimality Proofs

The sixteen formalizations in Table 1 contain about 377,000 lines of Agda code and use no postulates, and each theorem is as general as the result in its source paper. Moreover, all matrices are computed exactly. Their entries lie in algebraic number rings, whose elements are given by finitely many integers, so Agda checks automatically whether two matrices are equal. Over the real or complex numbers, no such automatic check exists, which limits the equality checking of EquaCert to discrete gate sets, as we discuss in Section 7.

Figure 6 adds completeness status to the families and inclusions of Figure 2, using the survey's notation [4]. It separates circuit relations from presentations of the underlying matrix groups. EquaCert covers the integral, scaled-real, Gaussian, and real matrix presentations, but circuit completeness for Clifford+T and Clifford+CS is still restricted to small widths. A presentation of the ambient matrix group does not by itself provide relations for every circuit subgroup.

The green checks in Figure 6 correspond to the qubit part of Table 1; the qudit results and minimality proofs are also listed in the table. The asterisks connect this coverage to the source corrections in Section 5.

Two qualifications apply to the Agda results in Table 1. The two-qubit Clifford+T and three-qubit Clifford+CS developments [15, 16] reuse the authors' Agda proofs and connect them to EquaCert's shared matrix presentations (Section 3). The qudit Clifford theorem of Bian, Li et al. [12] presents an independently constructed algebraic model; Appendix D.3 explains its connection to the intended unitary semantics.

4.1 Comparison of the Agda and Lean Formalizations

The Lean [30] version formalizes the same sixteen results using Mathlib's standard definitions [69], with matrix groups over real or complex numbers. It translates the Agda framework, so agreement between the two does not independently validate their shared design. Figure 7 compares proof size, verification time, and execution time. Lean developments generally use fewer lines and Agda records lower verification times in thirteen rows, but proofs and measurement methods differ. Translating the same proof, for Toffoli–Hadamard and for the four-bit theorem of Section 6, yields similar amounts of text. Lean takes 3.4 and 10 times Agda's time to check the two translations, in the second case mostly to load Mathlib once for each of its 54 modules. Agda provides executable normal forms, whereas Lean's are existence results or `noncomputable` definitions. Panel (c) measures their executable circuit-equivalence procedures, where Lean is faster from three qubits onward. Appendix C gives the detailed comparison and measurement methods, summarized in Table 2.

5 Corrections to the Source Papers

The formalizations in Table 1 identify two kinds of issues in the source papers. Most are typos, and some of them appear in statements that the proofs rely on. These are easy to fix. Two others are harder to repair. One is a gap in a proof, and the other is a relation missing from a theorem. We describe both below.

The gap is in the proof of Theorem 4.14 of Hadamard-Π [32], which states that the relations in its Figure 6, over the one- and two-level matrices $(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, and $\mathrm{H}_{[a,b]}$, present the group $O_n\left(\mathbb{Z}\left[1/\sqrt{2}\right]\right)$. One case of the proof relies on Lemmas A.16 and A.17, which have counterexamples in dimension 6, and the case analysis built on them misses a matrix in dimension 9. Their main result is however still correct: Clément's unpublished notes [24] present the same group with

Source	Object	Result	Agda Mod	Agda time (s)	Lean time (s)
Groups of m-level matrices					
Greylyn [2014]	$U_n(\mathbb{Z}[1/\sqrt{2}, i])$	presentation, $n \leq 4$	34	37.28	145.34
Bian and Selinger [2021]	$U_n(\mathbb{Z}[1/2, i])$	presentation	39	53.73	126.99
Li et al. [2021]	$O_n(\mathbb{Z}[1/2])$, L_n	presentations	71	84.52	185.32
Fang et al. [2026]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	46	48.33	231.57
Clément [2019]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	43	67.72	51.99
Circuits					
Selinger [2015]	Clifford	sub-presentation	32	35.01	88.24
Bian, Li et al. [2026a]	Clifford, odd prime p	presentation of a model	216	499.37	197.32
Makary et al. [2021]	real stabilizer	sub-presentation	31	37.94	79.75
Amy et al. [2018]	CNOT-dihedral	sub-presentation	31	21.24	101.22
Blake [2026b]	phase-affine, prime d	sub-presentations	57	83.30	257.24
Bian and Selinger [2023a]	two-qubit Clifford+T	sub-presentation in U_4	63	135.79	356.79
Bian and Selinger [2023b]	three-qubit Clifford+CS	sub-presentation in U_8	125	298.29	1330.89
Clément [2026]	real Clifford+CH	sub-presentation	348	419.99	5478.79*
Amy et al. [2024]	three-qubit Toffoli–Hadamard	presentation of L_8	203	111.17	134.91
Li et al. [2025]	qutrit Clifford	presentation	90	285.58	179.60
Minimality					
Blake [2026c]	six fragments	minimal, complete	80	180.10	358.87
Total			1,509	2,399.36	9,304.83

Table 1. Completeness and minimality results formalized in EquaCert. A presentation gives generators and relations for a whole group, and a sub-presentation for the subgroup generated by the gate set named in the Object column of the Circuits block. L_n is the group of orthogonal matrices $N/\sqrt{2}^j$, where N is an integer matrix and $j \geq 0$. Mod counts Agda modules. The Agda times are typechecking times from scratch with Agda 2.8, the times its profiler reports for the row's modules; the Lean times are elapsed folder-build times. Both were measured on the same machine, under the conditions in Appendix C. Both columns exclude the shared library and other developments on which a row depends. The Result column describes the Agda statements; Appendix C explains the semantic differences. *Includes two unused legacy helpers that failed; the main theorem chain passed.

different relations, and his case analysis covers the missing case, using which the formalization of Theorem 4.14 is completed. We also prove that the two sets of relations are equivalent in every dimension and formalize Clément's proof [6].

The relation is missing from Theorem 5.2 of [3]. It states that the relations in its Figures 4 and 7, over the Toffoli and $\mathrm{K} = \mathrm{H} \otimes \mathrm{H}$ gates, present the group $O_8(\mathbb{Z}[1/2])$. This is the first of two steps towards presenting the scaled dyadic orthogonal group L_8 by three-qubit Toffoli–Hadamard circuits. The relation r_{K} in its Section 5.5, which splits K on two qubits into two copies controlled by the third qubit, is in neither figure. Nor can it be derived from them. Hence, Theorem 5.2 fails as stated.

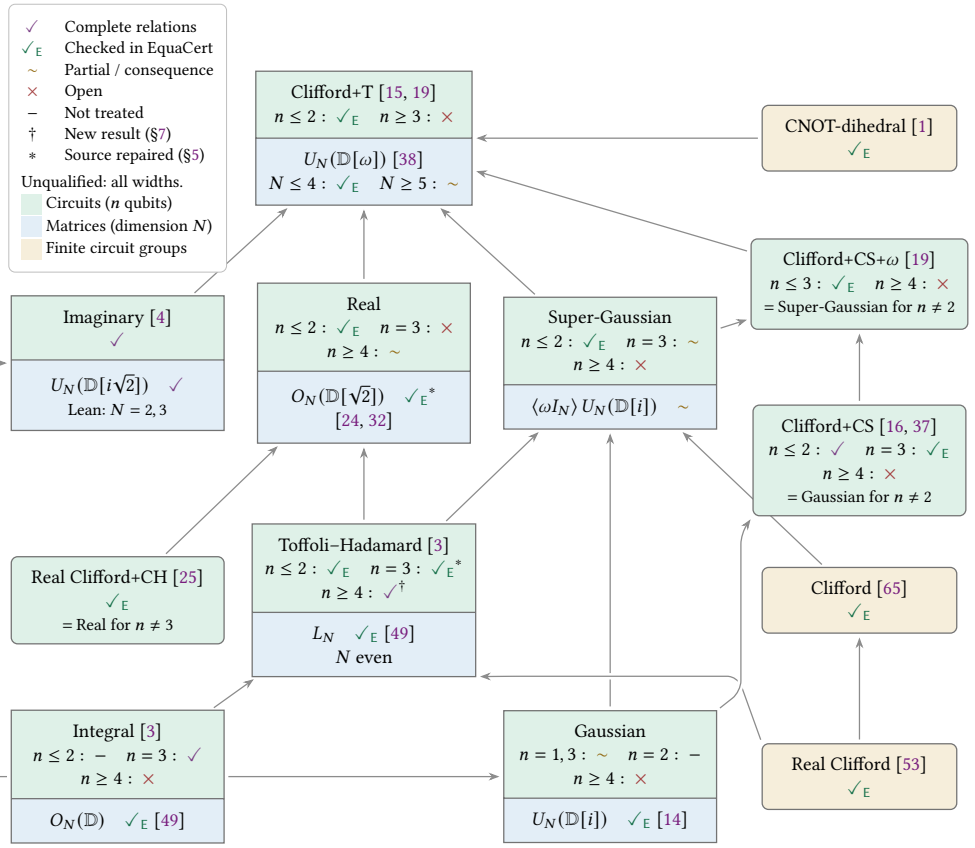


Fig. 6. Completeness progress on the same families and inclusion arrows as Figure 2. The dagger marks the new Toffoli–Hadamard result, whose exposition is in preparation. Gates and multi-level generators are omitted; upper compartments concern ancilla-free circuits, lower compartments matrix presentations. Here n counts qubits and N is matrix dimension ($N = 2^n$ for a circuit). The survey's $\sim$ includes consequences whose gate-set transfer is not formalized in the corpus; a plain check does not assert EquaCert verification. Imaginary-family checks refer to new computer-assisted proofs [4]; their complete Lean checks cover only $N = 2, 3$ and are outside this corpus. The higher-dimensional Clifford+T matrix presentation remains incomplete. L_N is defined in Table 1; the other notation is as in Figure 2.

Our formalization [7] shows that r_{K} is independent by giving an interpretation of circuits that every other relation preserves but r_{K} does not, and proves the presentations of O_8 ($\mathbb{Z}[1/2]$) and L_8 with r_{K} restored.

6 A New Result: Complete Equations for Four-Bit Reversible Circuits

Beyond the published results, we use EquaCert to prove and check a new completeness theorem for classical reversible circuits. Without ancillas, circuits over the gates X (NOT), CX (CNOT), and CCX (Toffoli) realize every permutation of the 2^k bit strings on $k \leq 3$ wires, and exactly the even ones on $k \geq 4$ wires [66]. Complete equational theories are known for linear reversible circuits [47, 63] and for circuits over multiple-control Toffoli gates [33, 34]. We give one for {X, CX, CCX} on at most four wires: the theory $\mathcal{T}$ of the seven equations in Figure 9.

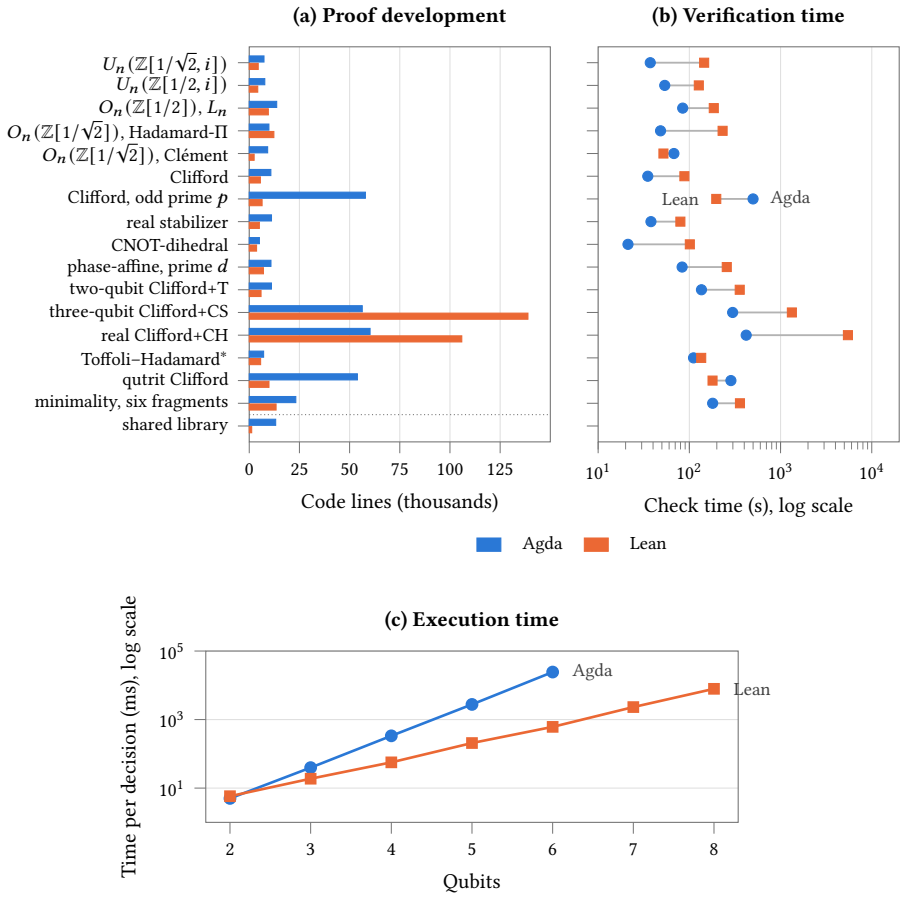


Fig. 7. The Agda and Lean formalizations of each row of Table 1. (a) Code lines, excluding blank lines and comments. For Toffoli–Hadamard* it shows another pair, one proof in both assistants (Appendix C): a second Agda proof and its translation to Lean. The last row is the shared library of each framework. (b) The check times of Table 1, Agda (circles) and Lean (squares), measured on the same machine, with methods described in Appendix C. (c) The time each compiled formalization takes to decide that two Clifford circuits of 100 gates have the same matrix, by the procedure its soundness proof uses, as medians over three runs of five random circuits.

Notation. Wires are numbered from 0 at the bottom, and a *state* of k wires is a string of k bits, that of the top wire first. A circuit is a product of the gates X, CX, and CCX and of swaps of adjacent wires, and a circuit u on k wires denotes a permutation $[\![u]\!]$ of the 2^k states. We write products of circuits and of permutations in diagrammatic order, meaning that pq applies p first and then q. Two circuits are *equal modulo* $\mathcal{T}$ if the equations of Figure 9 and the structural rules of Figure 10 rewrite one into the other. In circuit diagrams, time runs from left to right, $\oplus$ is a NOT target, crossed wires are the structural swap, and a slashed wire is a bundle of wires (Figure 8a). A filled control tests 1, and an open control tests 0 and stands for a filled control between two X gates on its wire (Figure 8b).



(a) A slashed wire is a bundle of wires.

(b) An open control is a filled control between two X gates on its wire.

Fig. 8. Two conventions of the circuit diagrams in Section 6.

A CNOT or Toffoli acts on adjacent wires with its target below its controls, and one drawn elsewhere is that gate conjugated by swaps. In Lafont's diagrams (Figure 11), each wire is instead a point and each grey box is a cycle. Following Lafont, ω_i is the cycle of the points i, $i+1$, and $i+2$, a stairs $s_d(j)$ is a product of consecutive cycles that carries d to j, and a canonical form is a product of stairs $s_d(j_d)$, one for each d, whose targets are the j_d. On four wires, the points are the sixteen states, which we number along the reflected Gray code and call positions. The circuit Ω_i carries out ω_i on the positions, and $\mathrm{Can}(j_0, \ldots, j_{13})$ is the canonical form with these targets, read as a circuit. In the Agda statements, $\vdash^{\mathsf{R}}$ denotes equality modulo $\mathcal{T}$, $\doteq$ equality of the functions $[\![w]\!]$ at every state, canon x the canonical circuit with the targets x, and nf w the tuple of targets of the permutation that w carries out (nf-sem of the module NormalForm).

Theorem 6.1 (soundness and completeness). *On any number of wires, two circuits that are equal modulo $\mathcal{T}$ denote the same permutation. Conversely, for $1 \le k \le 4$, two circuits on k wires that denote the same permutation are equal modulo $\mathcal{T}$, and the circuits on k wires modulo $\mathcal{T}$ form a group that $u \mapsto [\![u]\!]$ maps isomorphically onto the symmetric group $\mathrm{Sym}(2^k)$ for $k \le 3$ and onto the alternating group $\mathrm{Alt}(16)$ for $k = 4$.*

On four wires, every circuit also has a normal form: Lafont's canonical form of an even permutation [47], read as a circuit. Lafont generates the even permutations of N *points*, which we number $0, \ldots, N-1$, by the *cycles* $\omega_0, \ldots, \omega_{N-3}$, where ω_i carries i to $i+1$, $i+1$ to $i+2$, and $i+2$ to i (Figure 11(a)). The *stairs* from d to j, for $d \le N-3$, is the word $s_d(j) = \omega_d\, \omega_{d+1} \cdots \omega_{j-1}$ for $d \le j \le N-2$, and $s_d(N-1) = s_d(N-2)\, \omega_{N-3}$; it carries d to j and fixes the points below d. A *canonical form* is a product of stairs $s_{N-3}(j_{N-3}) \cdots s_1(j_1)\, s_0(j_0)$ with $d \le j_d \le N-1$ (Figure 11(b)). Every even permutation π is the permutation of exactly one canonical form [47, Lemma 6]; its *targets* $j_0, \ldots, j_{N-3}$ are found in turn: j_d is the image of d under $\pi\, s_0(j_0)^{-1} \cdots s_{d-1}(j_{d-1})^{-1}$.

Here the points are the sixteen states of four wires, numbered along the reflected Gray code: *position i* is the state with the binary digits of $i \oplus \lfloor i/2 \rfloor$, and a circuit *carries out* a permutation of the positions. Consecutive positions differ on one wire, so the cycle ω_i is carried out by a circuit Ω_i that is equal modulo $\mathcal{T}$ to a commutator of two Toffoli gates with filled or open controls (Figure 12(a), Appendices E.3 and E.8). Writing Ω_i for ω_i turns the canonical form with the targets $j_0, \ldots, j_{13}$ into the *canonical circuit* $\mathrm{Can}(j_0, \ldots, j_{13})$.

Theorem 6.2 (normal form). *Every four-wire circuit u is equal modulo $\mathcal{T}$ to exactly one canonical circuit, the one with the targets of the permutation that u carries out. Moreover, $(j_0, \ldots, j_{13}) \mapsto [\![\mathrm{Can}(j_0, \ldots, j_{13})]\!]$ is a bijection from the $16!/2$ tuples of targets onto $\mathrm{Alt}(16)$.*

For example, the Toffoli gate of Figure 12(b) exchanges the positions 4 and 5 and the positions 10 and 11. The canonical form of this permutation is $\omega_9\, \omega_8\, \omega_7\, \omega_6\, \omega_5\, \omega_4$, six stairs of one cycle each (Figure 11(c)), so the gate is equal modulo $\mathcal{T}$ to the canonical circuit $\Omega_9\, \Omega_8\, \Omega_7\, \Omega_6\, \Omega_5\, \Omega_4$ (Appendix E.11).

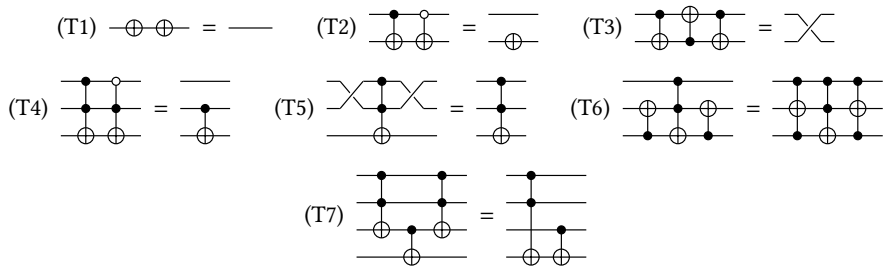


Fig. 9. The theory $\mathcal{T}$ for circuits over {X, CX, CCX}. Time runs left to right, and wire 0 is the bottom wire. A filled control tests 1, an open control tests 0 (it is the filled one conjugated by X on its wire), $\oplus$ is a NOT target, and crossed wires are the structural swap. A CNOT or Toffoli drawn in another position, as in (T3), (T6), and (T7), is the gate on adjacent wires with its target below its controls, conjugated by swaps. Each equation can be used in either direction and, by the structural rules of Figure 10, on any wires of a circuit, adjacent or not and in any order.

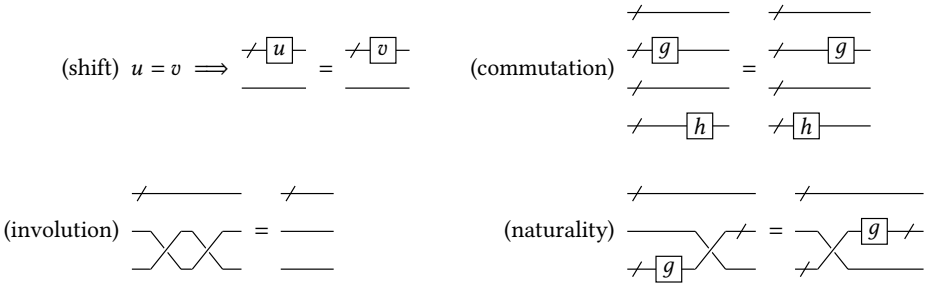


Fig. 10. The structural rules for circuits over {X, CX, CCX}, which EquaCert supplies for every gate set with a structural swap (Appendix B.1). Here u and v are circuits, g and h are X, CX, CCX, or the swap, and a slashed wire carries the wires of the box on it or, if it has none, any number of idle wires. An equation also holds with an idle wire below it, a gate commutes with any gate above it, the swap is an involution, and a gate slides past a wire that crosses all of its wires; for g the swap, (naturality) is the braid relation. (shift) adds wires only below, while the other rules hold under any idle wires.

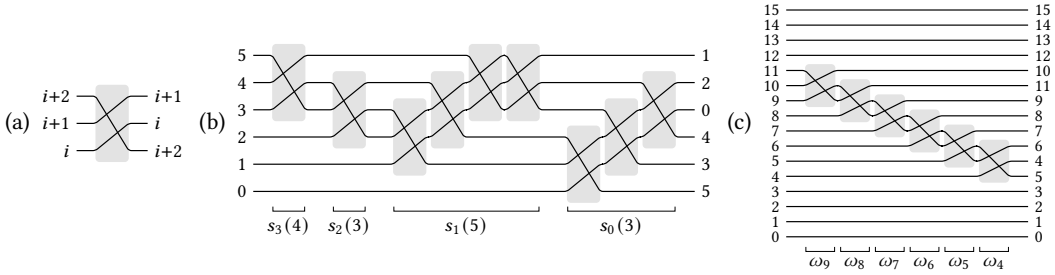


Fig. 11. Lafont's canonical forms as circuits whose wires are the points, after his drawings. Time runs left to right, the point 0 is the bottom wire, a grey box is a cycle, and each strand is labelled with its point at both ends, so it ends at the height of the image of its point. (a) The cycle ω_i. (b) The canonical form $s_3(4)\, s_2(3)\, s_1(5)\, s_0(3)$ on six points. The stairs $s_d(j)$ raises the strand of the point d to the height j, one wire at each cycle, and the stairs to the last point, $s_1(5)$, repeats its last cycle. (c) The canonical form $\omega_9\, \omega_8\, \omega_7\, \omega_6\, \omega_5\, \omega_4$ on the sixteen positions, which exchanges 4 with 5 and 10 with 11, as the Toffoli gate of Figure 12(b) does.

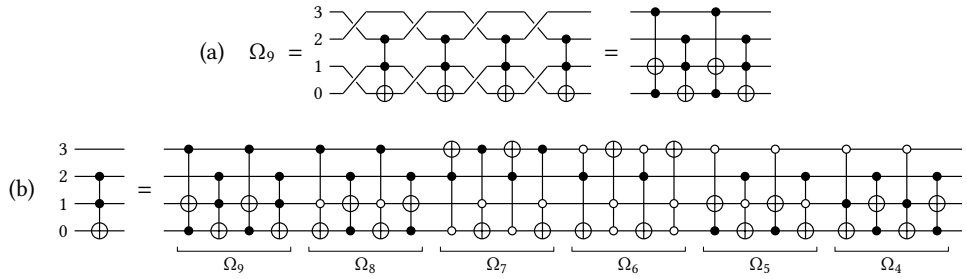


Fig. 12. The circuit Ω_9 and a canonical circuit, drawn as in Figure 9, with the wires numbered at the left. (a) By definition, Ω_9 is two swaps and then a Toffoli gate, repeated four times. Modulo $\mathcal{T}$ it is equal to a commutator of two Toffoli gates. (b) A Toffoli gate and the canonical circuit $\Omega_9\,\Omega_8\,\Omega_7\,\Omega_6\,\Omega_5\,\Omega_4$ that it is equal to modulo $\mathcal{T}$, the translation of Figure 11(c). Each Ω_i is drawn as a commutator of two Toffoli gates, with filled or open controls, that is equal to it modulo $\mathcal{T}$ (Appendix E.8).

Both theorems are proved in Agda, and Statements (6) to (10) are from the MainTheorem module of the formalization [5].

```
soundᴿ : ∀ {n} {w v : Circuit n} → ⊢ᴿ w ≈ v → [[ w ]] ≐ [[ v ]]                (6)
completeᴿ : ∀ {w v : Circuit 4} → [[ w ]] ≐ [[ v ]] → ⊢ᴿ w ≈ v                 (7)
presentationᴿ : (4 RRel,_===_) IsPresentationOf Alt-group 16                    (8)
normal-formᴿ : (w : Circuit 4) → ⊢ᴿ canon (nf w) ≈ w                            (9)
canon-unique : ∀ {x y : NF} → [[ canon x ]] ≐ [[ canon y ]] → x ≡ y             (10)
```

Proof idea. The proof (Appendix E) follows the second strategy of Figure 3. By two Reidemeister–Schreier steps [52], which the framework provides as a verified theorem (Section 3.2, Appendix E.1), it derives, from $\mathcal{T}$, Lafont's complete presentation of Alt(16) by the cycles [47]; the library of EquaCert proves that presentation and his lemmas on canonical forms. Read on circuits, his presentation becomes 111 equations: his 105 relations between the Ω_i, and six equations that write each gate and each swap as a product of the Ω_i. To derive them, $\mathcal{T}$ is first proved complete on at most three wires, from the Coxeter presentation of the symmetric group [29]. A four-wire derivation then consists of replacements of a circuit on three adjacent wires by one that denotes the same permutation, and of thirteen instances of the rules on all four wires, among them (T7). Nearly all derivations are found by search, and Agda replays the 15,607 steps of the four-wire ones in 20 seconds. Theorem 6.2 then follows from Lafont's lemmas. The formalization has 38 modules and about 28,000 lines, 24,000 of them generated derivations; Agda checks it in 45 seconds given the shared library, and it is translated to Lean module for module (Appendix C).

7 Discussion and Future Work

We formalize the results of sixteen papers in EquaCert using one circuit syntax, one definition of group presentations, and shared libraries for rings and their matrix groups. Unifying these definitions has two benefits. First, presentations of the same group from different papers can be compared and proved equivalent. Second, a new completeness proof can reuse much of the existing work. When completeness follows from an exact synthesis algorithm, the induction that turns the algorithm into a proof carries over with little change, so the main new work is the case analysis over matrices and generators. That case analysis is the most error-prone part of the proof, so it

is where machine checking matters most. Reusing these libraries, we also prove a new result, a complete equational theory for four-bit reversible circuits over {X, CX, CCX}.

We are now using EquaCert to extend two completeness results to any number of wires. The first is the four-bit theory of Section 6. For $n \geq 4$, circuits over {X, CX, CCX} realize exactly the even permutations [66], and related complete rule sets use cycles instead of gates [47] or allow multiple-control Toffoli gates [33, 34]. The second is the three-qubit Toffoli–Hadamard theory [3], which we extend using the presentation of $O_n(\mathbb{Z}[1/2])$ [49]. More broadly, completeness is still open for many other discrete gate sets, especially in higher dimensions, and we plan to study Clifford circuits in all dimensions with EquaCert. Beyond completeness, the exact synthesis libraries can be reused for T-count optimization of multi-qubit Clifford+T circuits.

At the moment, not every completeness result fits into EquaCert, as its scope is limited in two ways. First, it supports only discrete gate sets, so it does not cover completeness results for circuits with continuous parameters, such as rotations by arbitrary angles [26–28]. Second, control is not a circuit constructor in EquaCert, so it does not cover results that treat control as a primitive operation on circuits [17, 31, 64]. We plan to extend EquaCert past both limitations.

References

[1] Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A finite presentation of CNOT-dihedral operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97. doi:10.4204/EPTCS.266.5 arXiv:1701.00140

[2] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-theoretic characterizations of some restricted Clifford+T circuits. *Quantum* 4 (2020), 252. doi:10.22331/q-2020-04-06-252

[3] Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A sound and complete equational theory for 3-qubit Toffoli–Hadamard circuits. In *Proceedings of the 21st International Conference on Quantum Physics and Logic (QPL 2024) (Electronic Proceedings in Theoretical Computer Science, Vol. 406)*. 1–43. doi:10.4204/EPTCS.406.1 arXiv:2407.11152

[4] Anonymous. 2026. Complete relations for restricted Clifford+T operators: Status by exact synthesis algorithm. Unpublished working notes, version of 7 November 2026.

[5] Anonymous. 2026. EquaCert: Complete equations for four-bit reversible circuits from Lafont's presentation of the alternating group, formalized in Agda and Lean. Supplementary material, module `Alt16-RevCir/MainTheorem` (completeness on one to four wires, the presentations of $\mathrm{Sym}(2^k)$ and $\mathrm{Alt}(16)$, the two Reidemeister–Schreier steps, and the canonical form), and its Lean translation `QCert/Alt16RevCir/MainTheorem`.

[6] Anonymous. 2026. EquaCert: The presentations of $O_n(\mathbb{Z}[1/\sqrt{2}])$ by Hadamard-Π and by Clément, formalized in Agda. Supplementary material, modules `Real-Clifford+CH-TwoLevel/Equivalence` (the equivalence of the two sets of relations) and `Real-Clifford+CH-TwoLevel-Clement` (Clément's proof and its decision trees).

[7] Anonymous. 2026. EquaCert: Three-qubit Toffoli–Hadamard circuits with the relation r_{K}, formalized in Agda. Supplementary material, modules `CCX+H-3qubit-RS/Theorems` (the presentations of $O_8(\mathbb{Z}[1/2])$ and L_8) and `CCX+H-3qubit-RS/Independence` (the interpretation showing that r_{K} is not derivable).

[8] Anthropic. 2026. Formalizing Fermat's Last Theorem. Research post, 4 September 2026. https://www.anthropic.com/research/formalizing-fermats-last-theorem

[9] Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. 2026. Navier–Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. arXiv:2610.08144

[10] Maor Ben-Shahar, Jonathan Morag, Dirk Englund, and Kfir Sulimany. 2026. Quantum cryptography and autoformalization in Lean. arXiv:2610.17286

[11] Xiaoning Bian and Yuan Feng. 2027. Normal forms and complete relations for circuits in Agda. *Proceedings of the ACM on Programming Languages* 11, POPL (2027). To appear.

[12] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. arXiv:2609.40106

[13] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. Agda formalization. https://github.com/onestruggler/acir/releases/tag/qip-submission

[14] Xiaoning Bian and Peter Selinger. 2021. Generators and relations for $U_n(\mathbb{Z}[1/2, i])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 145–164. doi:10.4204/EPTCS.343.8 arXiv:2105.14047

[15] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 2-qubit Clifford+T operators. In *Proceedings of the 19th International Conference on Quantum Physics and Logic (QPL 2022) (Electronic Proceedings in Theoretical Computer Science, Vol. 394)*. 13–28. doi:10.4204/EPTCS.394.2 arXiv:2204.02217

[16] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 3-qubit Clifford+CS operators. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023) (Electronic Proceedings in Theoretical Computer Science, Vol. 384)*. 114–126. doi:10.4204/EPTCS.384.7 arXiv:2306.08530

[17] Colin Blake. 2026. A complete equational presentation of qudit circuits via polycontrolled PROPs. In *Proceedings of the 51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (Leibniz International Proceedings in Informatics, Vol. 386)*. 6:1–6:18. doi:10.4230/LIPIcs.MFCS.2026.6 arXiv:2602.09873

[18] Colin Blake. 2026. Completeness for prime-dimensional phase-affine circuits. arXiv:2603.06466

[19] Colin Blake. 2026. Simpler presentations for many fragments of quantum circuits. In *Proceedings of the 11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026) (Leibniz International Proceedings in Informatics, Vol. 378)*. 6:1–6:20. doi:10.4230/LIPIcs.FSCD.2026.6 arXiv:2602.09874

[20] Ana Bove, Peter Dybjer, and Ulf Norell. 2009. A brief overview of Agda – a functional language with dependent types. In *Proceedings of the 22nd International Conference on Theorem Proving in Higher Order Logics (TPHOLs 2009) (Lecture Notes in Computer Science, Vol. 5674)*. 73–78. doi:10.1007/978-3-642-03359-9_6

[21] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a few square roots, quantum computing is as easy as Π. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 29 pages. doi:10.1145/3632861 arXiv:2310.14056

[22] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An automated deductive verification framework for circuit-building quantum programs. In *Proceedings of the 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177. doi:10.1007/978-3-030-72019-3_6

[23] Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in reversible programming: From symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 6 (2022), 32 pages. doi:10.1145/3498667

[24] Alexandre Clément. 2019. A complete set of relations for the real Clifford+T group. Unfinished manuscript. https://www.mathstat.dal.ca/~aclement/Case%20distinction.pdf

[25] Alexandre Clément. 2026. A complete equational theory for real-Clifford+CH quantum circuits. arXiv:2602.06644

[26] Alexandre Clément, Noé Delorme, and Simon Perdrix. 2024. Minimal equational theories for quantum circuits. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. Article 27, 14 pages. doi:10.1145/3661814.3662088 arXiv:2311.07476

[27] Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024. Quantum circuit completeness: Extensions and simplifications. In *Proceedings of the 32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (Leibniz International Proceedings in Informatics, Vol. 288)*. 20:1–20:23. doi:10.4230/LIPIcs.CSL.2024.20 arXiv:2303.03117

[28] Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A complete equational theory for quantum circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. 1–13. doi:10.1109/LICS56636.2023.10175801 arXiv:2206.10577

[29] H. S. M. Coxeter and W. O. J. Moser. 1980. *Generators and relations for discrete groups* (4th ed.). Ergebnisse der Mathematik und ihrer Grenzgebiete, Vol. 14. Springer, Berlin. doi:10.1007/978-3-662-21943-0

[30] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 theorem prover and programming language. In *Proceedings of the 28th International Conference on Automated Deduction (CADE 2021) (Lecture Notes in Computer Science, Vol. 12699)*. 625–635. doi:10.1007/978-3-030-79876-5_37

[31] Noé Delorme and Simon Perdrix. 2026. Diagrammatic reasoning with control as a constructor, applications to quantum circuits. In *Proceedings of the 29th International Conference on Foundations of Software Science and Computation Structures (FoSSaCS 2026) (Lecture Notes in Computer Science, Vol. 16503)*. 240–261. doi:10.1007/978-3-032-22730-0_12 arXiv:2508.21756

[32] Wang Fang, Chris Heunen, and Robin Kaarsgaard. 2026. Hadamard-Π: Equational quantum programming. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 5 (2026), 27 pages. doi:10.1145/3776647 arXiv:2506.06835

[33] Shiguang Feng and Lvzhou Li. 2026. A complete set of transformation rules for reversible circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 8 (2026), 3711–3724. doi:10.1109/TCAD.2025.3641533 arXiv:2508.17273

[34] Shiguang Feng, Zongxing Xiong, and Lvzhou Li. 2026. On the completeness of transformation rules in reversible logic synthesis. arXiv:2607.19059

[35] Nikhil Garg. 2026. EconCSLib: AI-assisted Lean formalization for economics & computation research. arXiv:2606.13306

[36] Brett Giles and Peter Selinger. 2013. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A* 87, 3 (2013), 032332. doi:10.1103/PhysRevA.87.032332 arXiv:1212.0506

[37] Andrew N. Glaudell, Neil J. Ross, and Jacob M. Taylor. 2021. Optimal two-qubit circuits for universal fault-tolerant quantum computation. *npj Quantum Information* 7 (2021), 103. doi:10.1038/s41534-021-00424-z arXiv:2001.05997

[38] Seth Eveson Murray Greylyn. 2014. *Generators and relations for the group* $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. MSc thesis. Dalhousie University. arXiv:1408.6204

[39] David Gross. 2006. Hudson's theorem for finite-dimensional quantum systems. *Journal of Mathematical Physics* 47, 12 (2006), 122107. doi:10.1063/1.2393152 arXiv:quant-ph/0602001

[40] Shamgar Gurevich and Ronny Hadani. 2007. The geometric Weil representation. *Selecta Mathematica* 13, 3 (2007), 465–481. doi:10.1007/s00029-007-0047-3 arXiv:math/0610818

[41] Shamgar Gurevich and Ronny Hadani. 2010. Bounds on certain higher-dimensional exponential sums via the self-reducibility of the Weil representation. arXiv:math/0612765

[42] Thomas Hales. 2026. What mathematicians should know about the Lean theorem prover: Questions of reliability and AI. Guest post on Terence Tao's blog *What's new*, 9 October 2026. https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/

[43] Keming He, Jiahan Chen, Enji Xiong, Zhongyi Ni, Lei Zhang, and Xin Wang. 2026. QECert: Formal foundations for quantum error-correction theory in Lean. arXiv:2611.05047

[44] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318

[45] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. 2026. Lean-Quantum: Toward AI-assisted formalization of quantum information. arXiv:2607.05492

[46] Donald E. Knuth and Peter B. Bendix. 1970. Simple word problems in universal algebras. In *Computational Problems in Abstract Algebra*, John Leech (Ed.). Pergamon Press, Oxford, 263–297. doi:10.1016/B978-0-08-012975-4.50028-X

[47] Yves Lafont. 2003. Towards an algebraic theory of Boolean circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310. doi:10.1016/S0022-4049(03)00069-0

[48] Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A complete and natural rule set for multi-qutrit Clifford circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. doi:10.4204/EPTCS.426.2 arXiv:2508.14670

[49] Sarah Meng Li, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for the group $O_n(\mathbb{Z}[1/2])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 210–264. doi:10.4204/EPTCS.343.11 arXiv:2106.01175

[50] Sirui Lu, Ruixuan Deng, Yanqiao Zhu, and Zhengfeng Ji. 2026. Long-horizon autoformalization of a core theorem underlying MIP* = RE. arXiv:2609.19814

[51] Saunders Mac Lane. 1965. Categorical algebra. *Bulletin of the American Mathematical Society* 71, 1 (1965), 40–106. doi:10.1090/S0002-9904-1965-11234-4

[52] Wilhelm Magnus, Abraham Karrass, and Donald Solitar. 2004. *Combinatorial group theory: Presentations of groups in terms of generators and relations* (2nd revised ed.). Dover Publications, Mineola, NY.

[53] Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for real stabilizer operators. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 14–36. doi:10.4204/EPTCS.343.2 arXiv:2109.05655

[54] Dmitri Maslov, Gerhard W. Dueck, D. Michael Miller, and Camille Negrevergne. 2008. Quantum circuit simplification and level compaction. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 3 (2008), 436–444. doi:10.1109/TCAD.2007.911334

[55] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. 2018. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information* 4 (2018), 23. doi:10.1038/s41534-018-0072-4

[56] Markus Neuhauser. 2002. An explicit construction of the metaplectic representation over a finite field. *Journal of Lie Theory* 12, 1 (2002), 15–30. doi:10.5802/jolt.250

[57] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph. D. Dissertation. Chalmers University of Technology.

[58] OpenAI. 2026. Mathematical manuscripts and Lean formalizations from an internal OpenAI model. GitHub repository, released 6 October 2026. https://github.com/openai/math

[59] Zoe Paraskevopoulou. 2026. Machine-generated, machine-checked proofs for a verified compiler (experience report). arXiv:2602.20082

[60] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A core language for quantum circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858. doi:10.1145/3009837.3009894

[61] Yuanjie Ren, Jinzheng Li, and Yidi Qi. 2026. MerLean: An agentic framework for autoformalization in quantum computation. arXiv:2602.16554

[62] Yuanjie (Collin) Ren, Zhicheng Zhang, Junyi Liu, Jinzheng Li, Yidi Qi, and Mingsheng Ying. 2026. QEDIA: Quantum program verification in Lean. arXiv:2610.17708

[63] Issei Sakashita. 2013. A complete transformation rule set and a minimal equation set for CNOT-based 3-qubit quantum circuits (Draft). arXiv:1304.5916

[64] William Schober and Scott Wesley. 2026. A complete equational theory for quantum circuits with generalized control. Accepted to the proceedings track of the 23rd International Conference on Quantum Physics and Logic (QPL 2026). https://cqi.inf.usi.ch/publications/Hierarchical_QPL2026.pdf

[65] Peter Selinger. 2015. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science* 11, 2, Article 10 (2015). doi:10.2168/LMCS-11(2:10)2015 arXiv:1310.6813

[66] Vivek V. Shende, Aditya K. Prasad, Igor L. Markov, and John P. Hayes. 2003. Synthesis of reversible logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 22, 6 (2003), 710–722. doi:10.1109/TCAD.2003.811448

[67] Sergey Shirobokov, Ben Jaderberg, Katerina Hristova, George Tsoukalas, Johannes Bausch, Eric Wieser, Francisco J. H. Heras, Nicholas C. Rubin, Guang Hao Low, Sébastien Racanière, Nenad Tomašev, Lei M. Zhang, James Watson, Tanuj Khattar, Aws Albarghouthi, Stavan Jain, Harun Khan, Leopoldo Sarra, Andrew W. Senior, Michael Broughton, Hussain Masoom, Sergio Boixo, Ryan Babbush, Swarat Chaudhuri, and Pushmeet Kohli. 2026. Discovering formally verified quantum programs with AI. arXiv:2611.09822

[68] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. t|ket⟩: A retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 014003. doi:10.1088/2058-9565/ab8e92

[69] The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381. doi:10.1145/3372885.3373824

[70] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: Superoptimization of quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. 625–640. doi:10.1145/3519939.3523433

[71] Shigeru Yamashita and Igor L. Markov. 2010. Fast equivalence-checking for quantum circuits. *Quantum Information and Computation* 10, 9–10 (2010), 721–734. doi:10.26421/QIC10.9-10-1

[72] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*.

[73] Lei Zhang, Mingrui Jing, Yusheng Zhao, Jiahong Liu, and Xin Wang. 2026. Lean-QuantumAlg: A framework for AI agents to produce quantum algorithms that humans can trust. arXiv:2610.30216

[74] Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational verification of quantum programs. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 29 (2023), 33 pages. doi:10.1145/3571222

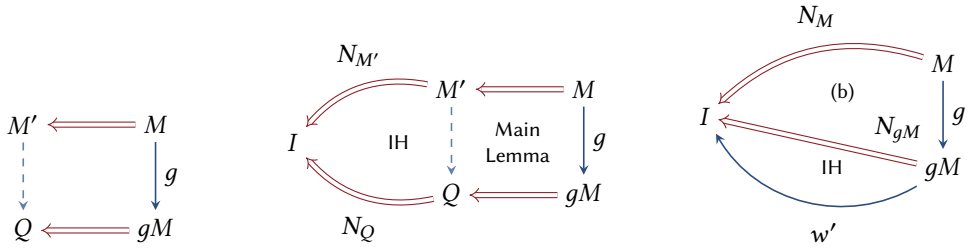
[75] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. 2026. Lean-QIT: Towards a formal infrastructure for quantum information theory. arXiv:2607.09632

A Completeness from Exact Synthesis

For the matrix groups discussed in Section 2, normal forms come from exact synthesis, and the proof that every circuit can be transformed into its normal form has the same structure for each algorithm. Each matrix, or *state*, has a *level*, built from quantities such as the least denominator exponent of a column, and the algorithm reduces each state M to the identity in steps that lower the level. These steps form the normal word N_M.

The *Main Lemma* says that for every state M and generator g, a diagram of relations closes the square formed by g and the algorithm's next steps from M and gM, passing only through states of lower level (Figure 13a).

By well-founded induction on the level, the relations then rewrite $N_{gM}\, g$ into N_M (Figure 13b), and by induction on length, every word that reduces M to the identity rewrites into N_M (Figure 13c). Two words with the same matrix therefore rewrite into each other. These inductions have the same form for every presentation, so the main work in each formalization is its Main Lemma.



(a) The Main Lemma. (b) Induction on the level. (c) Induction on length.

Fig. 13. Completeness from the Main Lemma. Plain arrows are words, double arrows are steps or runs of the algorithm, and dashed arrows are words through states of lower level.

B EquaCert Implementation Details

This appendix gives the circuit encoding, theorem notation, and matrix equality procedure used by the framework in Section 3.

B.1 Wire Placement and Structural Rules

Wires are numbered from 0 at the bottom, and a gate acts on the bottom wires of a circuit. A shift places a circuit on n wires onto wires $1, \ldots, n$ of a circuit with $n + 1$ wires. In the matrix semantics, both placing a gate in a larger circuit and shifting a circuit are Kronecker products with an identity. This defines wire placement once for all gate sets.

Each source relation is stated on as many wires as the paper draws it on. Structural rules allow an equation to be shifted by one wire, gates on disjoint wires to commute, and a scalar to be placed on any wire. EquaCert extends the earlier framework's one- and two-wire syntax with three-wire gates and their structural rule. Scalars are zero-wire generators; their placement rule allows exact equations to account for global phase.

In the earlier framework, swap is an ordinary gate whose laws must be included in each rule set. EquaCert also supports a PROP presentation, in which a designated two-wire gate is the swap and the framework supplies its involution law and the laws for moving gates across it. The source rule set then contains only its own relations, each on its original wires; swap and its laws are supplied by the framework. We use this representation for the circuits of [1, 18, 19], and for the reversible circuits of Section 6, whose structural rules are drawn in Figure 10.

B.2 Theorem Notation

The signatures in Section 3 use `n CRel,_===_` for relations on circuits with n wires and `n MRel,_===_` for relations on words over multi-level matrices of dimension n. In Statement (1), the former contains R1–R16 of Figure 1 of [53], lifted to n wires. In Statement (2), the latter contains the relations of Figure 6 of Hadamard-Π [32].

The judgement `⊢ w ≈ v` states that w and v are equal by the relations. The qubit developments interpret matrices in $\mathbb{D}[\omega]$ and use `_≈ᵁ_` for their equality; the qudit developments compare interpretations in their own groups. The `IsSubPresentationOf` record contains the semantics and its soundness and completeness proofs, and `IsPresentationOf` adds surjectivity.

Minimality is stated for the rule set `n ⊢_===_` in Statement (5). Unlike the generated equivalence relation, this retains the individual rules, so removing one is defined. The predicate `IsMinimal` therefore takes this rule set rather than its generated equivalence relation.

B.3 Matrix Equality Checking

The main computational cost came from Agda's conversion checker. It accepts syntactically identical terms immediately; otherwise, it evaluates them to compare their results. For long matrix products, this evaluation can take exponential time. We keep circuit interpretations abstract and decide matrix equality by computing a single Boolean using row operations. For these products, this is orders of magnitude faster than comparing matrices entry by entry. These techniques are included in the agents' instructions so that later formalizations avoid the same slowdowns.

C Details of the Agda and Lean Comparison

The Lean [30] version of EquaCert formalizes the same sixteen results on top of Mathlib [69]. Once the Agda formalization exists, translating it to Lean is comparatively easy, so the Lean version is not a second achievement of the same kind. Its point is its theorem statements. They rest on Mathlib's standard definitions, with matrix groups over Mathlib's real or complex numbers in place of our exact rings, so a reader can check them against the source papers using definitions that are already widely used and reviewed. For qudit Clifford, the Lean statement also removes an argument that the Agda statement leaves to external mathematics (Section D.3).

Nor is the Lean framework an independent implementation. We instructed the coding agent that wrote it not to consult our Agda implementation, but the Lean framework is nonetheless a translation of the Agda one. Its words, circuit generators, structural rules, and presentation and sub-presentation predicates are those of the Agda framework, down to the names of the definitions and the wording of their documentation. The two formalizations therefore share the design of their framework, and would share any mistake in it, so their agreement is weaker evidence than two independent formalizations would give. They use different representations and sometimes different proofs, so the theorem statements of each must be checked against the source papers separately. We compare their definitions, proof development, verification time, and execution time; Table 2 summarizes the comparison. These observations concern our two formalizations, not Agda and Lean in general.

Definitions and Semantics. The translation keeps the definitions but adapts their representation to Mathlib. Agda uses its own word type, with explicit brackets, and the setoid-based algebra of the standard library. A presentation is a record containing a semantics and a group isomorphism, or a monomorphism for a sub-presentation. Lean uses Mathlib's free monoid and the monoid congruence generated by the relations. Its presentations are predicates in `Prop`, from which it

	Agda	Lean
Proof development		
Logic	constructive: every module `--safe`, no postulates	classical: every main theorem uses `Classical.choice`
Library	standard library (104 modules imported), EucDomain	Mathlib (56 modules imported directly)
Proofs	terms, by pattern matching and equational reasoning; reflection-based solvers	tactics: about 75,000 tactic blocks, 69,000 automation calls
Reused from the sources	the authors' Agda proofs for Clifford+T and Clifford+CS	no proofs; the authors' coset-action data for Clifford+T
Modules	1,509	795
Code lines	376,627	338,866
Verification time		
Decided equalities	a Boolean reduced to `true`	`decide +kernel`
Check time, sixteen rows	2,399 s	9,305 s
Faster for	thirteen rows	three rows
Execution time		
Normal forms	executable functions	existence theorems or `noncomputable` definitions (460 declared)
Benchmarked procedure	matrix of each circuit, by products of stored matrices	simulation of each circuit on every basis vector
Compiled through	GHC (Haskell)	C
Per decision, Clifford, 100 gates	5.0 ms on 2 qubits, 24 s on 6	5.8 ms on 2 qubits, 0.61 s on 6

Table 2. The Agda and Lean formalizations compared. Counts are over the files under version control in the sixteen development folders of Table 1, without the shared libraries, and code lines exclude blank lines and comments. Check times are those of Table 1, and execution times those of Figure 7(c).

derives isomorphisms out of the quotient. In both, the circuit infrastructure, the structural rules, and general completeness arguments are shared between developments.

The semantic targets also differ. Most Agda developments work directly with exact representations of matrix entries. Lean states its matrix groups over Mathlib's real or complex numbers and proves bridges to them from computable exact rings. In particular, the Agda qudit Clifford development presents the constructed model of Appendix D, whereas Lean proves soundness and completeness against complex matrices for every odd prime. A common source therefore does not imply identical statements or intermediate lemmas.

Proof Development. Excluding blank lines and comments, the Agda formalization of Table 1 has about 377,000 lines and the Lean formalization about 339,000 (Table 2). Figure 7(a) compares them row by row, except that for Toffoli–Hadamard it shows another pair, one proof in both assistants (see below). Lean's is shorter in thirteen of the sixteen rows, by a factor of five or more for qudit

Clifford and qutrit Clifford, where the two take different routes. It is longer for Hadamard-Π, three-qubit Clifford+CS, and real Clifford+CH. Both counts include the derivations and certificates that scripts generate. The Agda proofs are explicit terms, built by pattern matching and equational reasoning, with reflection-based solvers for associativity and ring identities. Its library develops the group theory the proofs need, in about 13,200 lines against 1,200 in Lean. The Lean proofs rely on Mathlib's algebra and on tactics, with about 69,000 calls to automation such as `simp`, `omega`, `decide`, and `ring`.

Both implementations reuse normal-form, exact-synthesis, and translation arguments. Both also check generated derivations with a replay procedure that is proved correct. The proof assistant checks the computation and the theorem that a successful replay gives a derivation, so the scripts that search for derivations need not be trusted. For Clifford+CS, Lean constructs its own coset certificates. The trust checks reflect the different foundations. Every Agda module is `--safe` and uses no postulates, so the Agda corpus is constructive. The Lean corpus uses no `native_decide`, and its main theorems depend on no unfinished proof. An audit of 105 declarations reports only the axioms `propext`, `Classical.choice`, and `Quot.sound`.

Verification Time. Table 1 and Figure 7(b) give the check times, both measured on the same machine. In total Agda took 2,399 seconds and Lean 9,305, or 1,979 and 3,826 without real Clifford+CH, whose Lean build alone took 5,479 seconds, more than half of the total. Lean took between 1.2 and 4.8 times as long as Agda on twelve rows, and 13 times as long on real Clifford+CH. Its five-wire certificate modules `Family267` and `Family268` alone took about 1,741 and 1,369 seconds. Lean was faster for three rows, Clément's presentation, qudit Clifford, and qutrit Clifford, whose Lean proofs take different routes. Both assistants decide equalities of matrices and certificates by evaluation in the checker. Agda reduces a Boolean to `true`, and Lean uses `decide +kernel`.

Both columns were measured under WSL on an Intel Core i5-13400TEF with sixteen logical CPUs, about 26 GiB of RAM, and 64 GiB of swap. The Agda column comes from a check from scratch on 8 November 2026, with Agda 2.8.0 and version 2.4 of its standard library. We removed the repository's interface files, kept those of the standard library and EucDomain, and checked the shared library and then each row in an Agda process of its own, through a module that imports every module of the row. Agda's profiler reports the time spent on each module without its imports, and a row's time is the sum over its own modules, wherever they were checked. It leaves out process startup and the loading of interface files: the whole check, shared library included, took 3,210 seconds of wall time, and no process used more than 5.1 GiB. No other proof assistant ran during the check. The Lean column comes from a clean build on 5–6 November 2026, with Lean 4.30.0 and the pinned Mathlib revision. We removed EquaCert's build artifacts, kept the Mathlib and other dependency caches, and built the folders one at a time in dependency order with `LEAN_NUM_THREADS=2`. Each time is the elapsed wall time of a folder, including Lake startup and artifact generation, and a shared dependency is counted only in the folder that defines it. The two columns therefore measure different things, per-module checking time in Agda and folder build time in Lean. Proof constructions, module boundaries, parallelism, and artifact generation also differ, so the differences cannot be attributed to the proof assistants alone. We have yet to separate the time Lean spends running automation from the time it spends checking the resulting proofs.

The build covered all 810 tracked library modules, excluding three generator templates, and took **9,415.37 seconds (2 h 36 min 55 s)** in timed build commands. Of these modules, 808 passed. Two unused legacy helpers in real Clifford+CH failed, `ItemBFun` at `omega` proofs and `ItemASteps` at the heartbeat limit. Neither is imported by the main theorem chain or the audit, both of which passed, and the real Clifford+CH entry of Table 1 includes the whole attempted build. Beyond the sixteen development folders, the shared `Circuit`, `Exact`, `Matrix`, and `NumberTheory` folders

took 17.91, 15.51, 6.88, and 15.29 seconds, and the five root modules, including the audit, took 54.95 seconds. Lean builds of another project ran on the same machine during part of the run. We therefore re-timed the twenty folders and root modules they overlapped and report the second timing, during which the other project used about a quarter of one CPU on average. The two runs total 9,604 and 9,415 seconds, and the two timings of each development folder agree within 15%, except for Toffoli–Hadamard (178 and 135 seconds).

One Proof in Both Assistants. For Toffoli–Hadamard, the formalizations of Table 1 have 28,629 lines in Agda and 4,401 in Lean, and they prove the result by different routes. The Agda proof replays the derivations in the authors' own files as generated certificates, which make up 22,572 of its lines. The Lean proof was written from the paper, over an alphabet of macro gates, with a Knuth–Bendix system for the symmetric group S_8 that the kernel checks. To compare the two assistants on one proof, Figure 7(a) shows another pair instead (∗). We wrote a second Agda proof of the result, which finds its circuit derivations by a breadth-first search over the relations of Figure 4 of [3], and a coding agent translated it to Lean module for module, keeping its definitions, derivations, and proof route; this time we told the agent to follow the Agda. A coding agent also began translating the first Agda proof in the same way, porting its scripts so that they write the certificates in Lean, and we stopped it after 46 of the 203 modules: the relations, the 218 derivations replayed from the authors' files, the 642 certificates of the permutation fragment, and the frames built on them, 17,862 of the Agda proof's lines. On one proof the two assistants need about the same amount of text: 182,301 non-blank characters outside comments in Agda and 191,589 in Lean for the second proof and its translation, and 1,056,950 and 1,030,864 for the translated modules of the first. In lines the translation of the second proof is shorter, 5,672 against 7,161, only because its generated derivations put several rewrite steps on a line where Agda's put one, while the translated modules of the first, whose certificates take a step per line in both, are 6% longer, 18,946 against 17,862. The hand-written part of the second proof's translation is the larger of its pair, mostly because it proves the facts about S_8 that the Agda proof imports from the shared library. Lean took 164 seconds to check it and Agda 48, 3.4 times as long, while on the two proofs of Table 1, timed in the same way, Lean took 118 seconds and Agda 111, 1.1 times as long (Table 1, timing Lean's whole build, has 135 and 111). So the pair that Table 1 and Figure 7(b) report reflects the difference between the proofs more than the difference between the assistants.

The new result of Section 6 gives a third pair. Coding agents translated the 38 Agda modules of its proof [5] to Lean module for module, again told to follow the Agda, and ported the scripts that write its generated derivations, so that they write Lean from the same search results. Sixteen further Lean modules translate what the Agda imports from the shared library: Lafont's presentation of the alternating groups with his canonical forms, the Coxeter presentation of the symmetric groups, the swap rules, and the morphisms between presentations. Again the two need about the same amount of text: 462,926 non-blank characters outside comments in Agda and 536,855 in Lean, or 484,418 without the sixteen library modules, in 24,062 and 23,083 lines. Lean took 288 seconds to check the translation and Agda 28, ten times as long, but most of Lean's time does not go to the proofs. Every Lean module first loads Mathlib, which takes about four seconds for a module with no content, and the generated derivations are fourteen modules that import one another in a chain, as in the Agda. The 54 modules took 354 seconds in all, some of them at the same time, so about 220 of these went to loading, and what is left is between four and five times Agda's time. Reaching that needed two departures from the Agda, both forced by the cost of evaluation in Lean's kernel. Applying one gate to one state, as the Agda semantics defines it, costs the kernel about half a millisecond, so the translation decides equalities of circuits on states encoded as numbers, after checking the arithmetic of each gate against the semantics once; this took its semantics module from 79 seconds

to 10. And where each step of an Agda derivation carries its own evidence, which Agda fills in by evaluation, the translation evaluates the evidence of all the steps of an equation in one kernel check, 788 checks for the 15,607 steps on four wires. These six times were measured on 10 November 2026 on the machine of Table 1, by its method, with no other check running.

Execution Time. The Agda normal forms are executable functions. The Lean developments obtain normal forms either as existence theorems or as `noncomputable` definitions. For example, the Lean proofs for the Clifford and real stabilizer circuits of [53, 65] show that every circuit equals a normal form without defining a function that computes it, and the exact synthesis of the two-level groups is noncomputable. In all, 460 definitions in the sixteen Lean developments are declared `noncomputable`, so there is no Lean normalizer to time. Both formalizations can, however, execute the procedure that decides whether two circuits have the same matrix, which their soundness proofs run. We compiled this procedure for the Clifford circuits of [65] to native code, through Agda's GHC backend and Lean's C backend. We then timed both on an Intel Core i5-14400, on the same random circuits of 100 gates over H, S, and CZ, deciding that each circuit w has the matrix of $w\,\mathrm{S}^4$ (Figure 7(c)). On two qubits Agda is slightly faster, at 5.0 against 5.8 milliseconds per decision. From three qubits on Lean is faster, by a factor of 2.1 on three qubits and 40 on six, where Agda takes 24 seconds and Lean 0.61. The gap comes mostly from the procedures rather than the compilers. Agda computes each circuit's matrix as a product of stored $2^n \times 2^n$ matrices, so its work grows about eightfold with each qubit. Lean simulates the circuit on each basis vector, so its work grows about fourfold.

D Case Study: Multi-Qudit Clifford Completeness

The multi-qudit Clifford result of Bian, Li et al. [12] shows how EquaCert turns a circuit completeness proof into reusable, checked components. Here each qudit has odd prime dimension p; the result covers every number n of qudits and every such p. Its sixteen rules must derive every valid equation between Clifford circuits. It illustrates two design choices of Section 3.2, defining the semantic group independently of the circuit syntax and assembling completeness proofs from general theorems about groups, as Figure 5 summarizes.

D.1 From Circuit Equations to Checked Derivations

The circuits use the Fourier gate H, the phase gate S, and the controlled-phase gate CZ. We first work projectively, identifying operators that differ only by a global phase. Figure 14 gives a small example of the translation into EquaCert: a swap is a word in the generators, its laws are circuit equations, and a derived equality records the rule used at each step. The framework supplies the structural rules that let the same derivation apply inside a larger circuit. The corresponding Agda definitions and proof terms are in Appendix D.4.

To prove completeness, we must also specify what a circuit means. We first consider its action on Pauli operators, ignoring phase factors. Each wire contributes a pair (a, b) modulo p, describing $\mathrm{X}^a\mathrm{Z}^b$ for the generalized Pauli shift and phase gates X and Z. The induced transformations form the symplectic group $\mathrm{Sp}(2n, \mathbb{Z}_p)$, which preserves Pauli commutation. This first stage identifies Clifford operators that differ by a Pauli; the next stage restores that information. This group is defined using linear algebra, independently of circuits and their rewrite rules. Completeness at this stage therefore requires a proof that equal transformations give derivable equations in the symplectic theory.

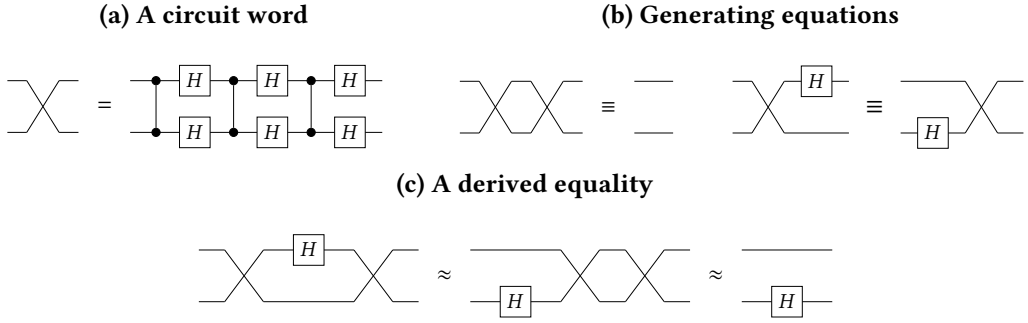


Fig. 14. Circuit syntax, rules, and reasoning in the projective Clifford formalization. Crossed wires denote swap, and connected dots denote CZ. Diagrams are read from right to left. In (c), moving H across one swap and cancelling the two swaps moves H to the other wire. The generating equations (≡) justify the derived equivalence (≈).

D.2 Normalize the Factors, Then Compose

The first step is a normal-form proof for this symplectic group. Figure 15 shows the recursive shape: a canonical block accounts for one wire, and normalization continues on the remaining wires. The formalization proves that every circuit rewrites to this shape, that each rewrite preserves its meaning, and that the normal form is uniquely determined by that meaning. Consequently, circuits denoting the same transformation rewrite to the same normal form and are equivalent. This is the normalization strategy of Section 2, instantiated with the source paper's circuit blocks; the datatypes and normalization table are in Appendix D.5.

Symplectic transformations retain only the Clifford action modulo Paulis. To restore the Paulis, we present their phase space $V = \mathbb{Z}_p^{2n}$ separately and combine the two presentations into the independently defined model $V \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. The semidirect product records how a symplectic transformation acts on Pauli exponents. EquaCert's general composition theorem asks us to prove that this action respects the relations of both factors. Once these obligations are checked, completeness of the combined presentation follows from completeness of the factors.

This replaces the most difficult part of following the source proof directly. Its passage from symplectic to projective completeness tracks a Pauli correction through each rewrite. We instead discharge the compatibility obligations once and use the framework's semidirect-product theorem. Verified translations then recover the paper's generators and relations. Finally, the scalar-extension theorem restores global phases, with the necessary phase corrections checked explicitly (Figure 5). The construction-specific calculations remain in the case study; the composition theorems can be reused by other developments.

D.3 What the Formalization Establishes

Agda checks the presentations of the symplectic, projective, and scalar-extension models for every odd prime p and every width n, with no postulates or unfinished proofs. In particular, it checks both normalization and the translations back to the source rule sets. This demonstrates how EquaCert can support a proof that needs substantial mathematical restructuring while keeping each obligation explicit.

The semantic boundary is also explicit. The Agda development presents independently constructed algebraic models; identifying them with the unitary Clifford groups and matching the paper's gate phases uses external mathematics. The Lean development proves soundness and

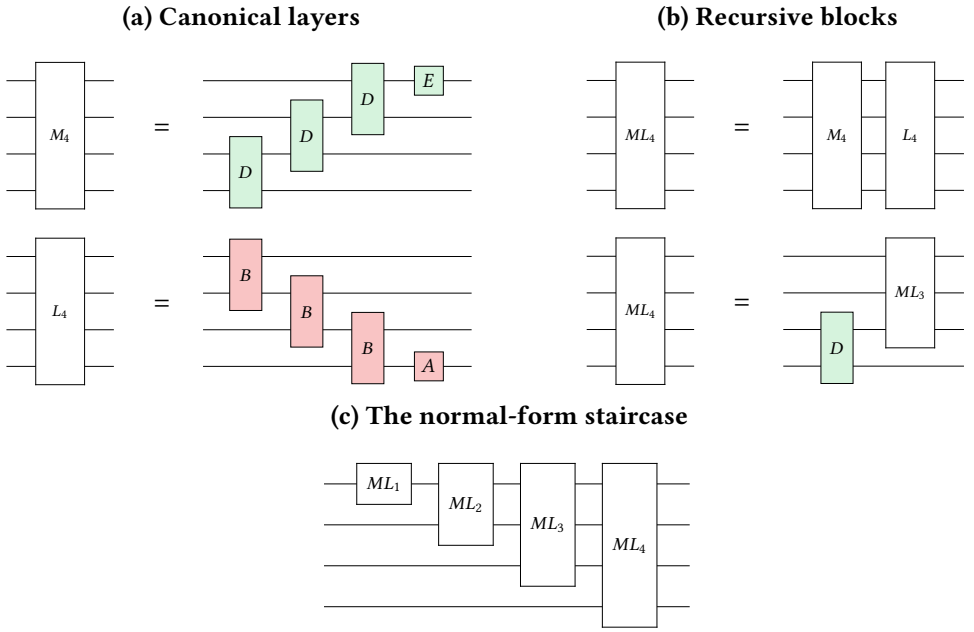


Fig. 15. The symplectic normal form, illustrated on four wires. (a) The M and L layers use the source paper's parameterized circuit blocks A, B, D, and E. (b) An ML block combines the two layers or extends a smaller ML block by a D block. (c) One block per width gives the staircase. Its uniqueness is the key completeness obligation; Paulis and scalar phases are restored separately.

completeness directly against complex matrices (Appendix C). The rest of this appendix gives the details. We first fix the gate conventions and state the formal theorems (Appendix D.4). We then prove symplectic completeness by normalization (Appendix D.5), restore Paulis and recover the source rule sets (Appendix D.6), and restore scalar phases (Appendix D.7). Appendix D.8 records the implementation and the boundary between the Agda proof and the external identification with unitary groups. Thus the case study shows both how the proof is assembled and exactly which connection each proof assistant checks.

D.4 Gate Conventions, Syntax, and Theorem Statements

We use the gate conventions of Bian, Li et al. [12]. Fix an odd prime p and let $\omega = e^{2\pi i/p}$. On the basis $|0\rangle, \ldots, |p-1\rangle$ indexed by $\mathbb{Z}_p$, the gates $\mathrm{X}|j\rangle = |j+1\rangle$ and $\mathrm{Z}|j\rangle = \omega^j|j\rangle$ generate, with the phases ω^t, the Pauli group $\mathcal{P}_n$ on n qudits. The gates

$$\mathrm{H}\colon |j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk}|k\rangle, \qquad \mathrm{S}\colon |j\rangle \mapsto \omega^{j(j-1)/2}|j\rangle, \qquad \mathrm{CZ}\colon |j\rangle|l\rangle \mapsto \omega^{jl}|j\rangle|l\rangle$$

generate, with the same phases, the Clifford group C_n. Here $\delta_p = (-i)^{(p-1)/2}$, which makes the determinant of H equal to 1. Modulo scalars, we write $\overline{C}_n = C_n/\langle\omega\rangle$ and $\overline{\mathcal{P}}_n = \mathcal{P}_n/\langle\omega\rangle \cong \mathbb{Z}_p^{2n}$. Then $C_n/\mathcal{P}_n \cong \overline{C}_n/\overline{\mathcal{P}}_n \cong \mathrm{Sp}(2n, \mathbb{Z}_p)$ and $\overline{C}_n \cong \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. We do not formalize these last two isomorphisms, because our semantic groups are *defined* as their right-hand sides. Bian, Li et al. [12] also include -1 among the global phases, so that the swap and the multiplier gates belong

to C_n. We work mainly in the projective quotient, where -1 makes no difference and the relations are slightly simpler.

Using the framework of Section 3, we encode circuits, rules, and reasoning with the rules. Figure 14 shows the diagrams for the following encodings. We draw circuits in matrix-multiplication order, the reverse of the usual circuit order, so $w \bullet v$ applies v first but draws it on the right. For example, this is the swap:

```
Ex : Word (Gen (2+ n))
Ex = CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑
```

A rule set is an inductive family indexed by the width, with one constructor per rule. Each rule is stated once, on the bottom wires of every width at which it fits. For example, these are two of the fifteen constructors of our primary rule set _SRel,_===_, the order of the swap and the rule moving H across it:

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order-Ex     : (2+ n) SRel, Ex ^ 2 === ε
  semi-Ex-H↑ : (2+ n) SRel, Ex • H ↑ === H ↓ • Ex
```

Reasoning with the rules is equational reasoning in the congruence $\approx$ they generate, with combinators for symmetry, associativity, and rewriting the left or right factor of a product. For example, conjugating by the swap moves H down a wire:

```
conj-Ex-H↑ : Ex • H ↑ • Ex ≈ H ↓
conj-Ex-H↑ = begin
  Ex • H ↑ • Ex   ≈⟨ sym assoc ⟩
  (Ex • H ↑) • Ex ≈⟨ cleft axiom semi-Ex-H↑ ⟩
  (H ↓ • Ex) • Ex ≈⟨ assoc ⟩
  H ↓ • Ex • Ex   ≈⟨ cright axiom order-Ex ⟩
  H ↓ • ε         ≈⟨ right-unit ⟩
  H ↓ ■
```

Theorem D.1 (Paper-V1.presentation). *For every width n, the fifteen rules of* `Paper-V1` *(Section D.6) present the projective Clifford model:*

```
presentation : ∀ n → (PapV1.Clifford-Relations._QRel,_===_ n) IsPresentationOf (Pauli⋊Sp n)
```

These rules are the relations of Bian, Li et al. [12] read modulo scalars, with its rule (C4) in an equivalent form. The relation _QRel,_===_ closes _SRel,_===_ under the structural rules (Section 3), and Pauli⋊Sp n is the semidirect product of $\mathbb{Z}_p^{2n}$ by $\mathrm{Sp}(2n, \mathbb{Z}_p)$ (Section D.6). The record _IsPresentationOf_ (Section 3) gives a group isomorphism from circuits modulo $\approx$ to $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. That this map is well defined is *soundness*, since each rule holds. That it is injective is *completeness*, and that it is surjective says that every element of the model is represented by a circuit.

We refine the normal form of Bian, Li et al. [12] for symplectic operators, which are Clifford operators modulo Paulis, into a doubly inductive definition (Section D.5), and we prove that normal forms are unique:

Theorem D.2 (unique-nf). *Normal forms with equal symplectic denotations are equal:*

```
unique-nf : ∀ n {u v : NF n} → [[ ⌈ u ⌉ ]] ≈ˢ [[ ⌈ v ⌉ ]] → u ≡ v
```

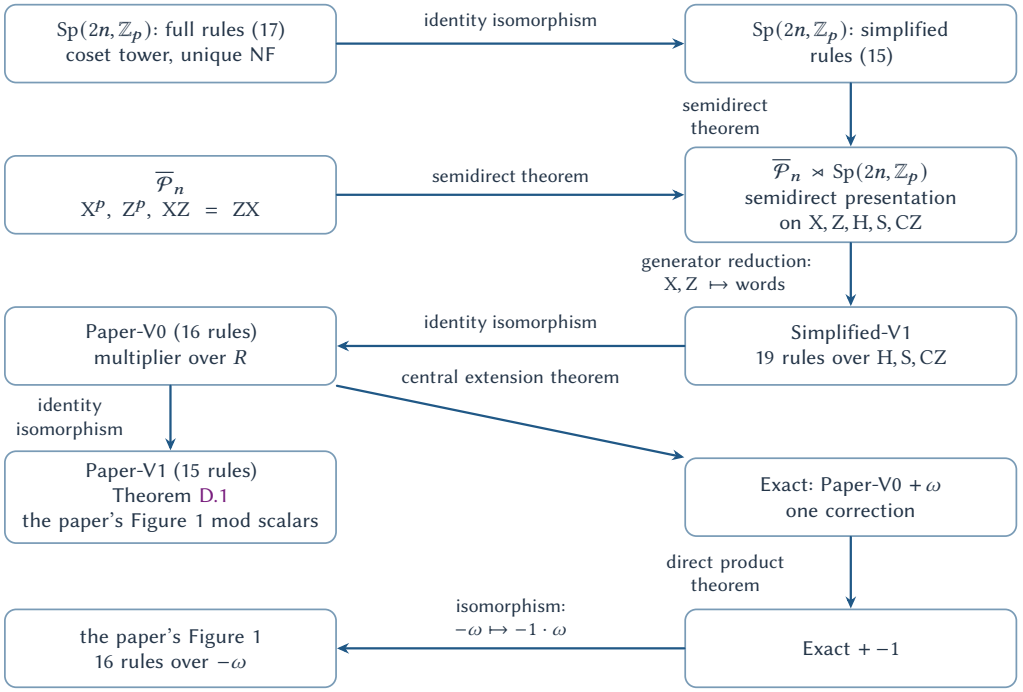


Fig. 16. The chain of presentations used to formalize the qudit Clifford result of Bian, Li et al. [12]. Every arrow is a verified presentation theorem or isomorphism of presentations. The symplectic stages present $\mathrm{Sp}(2n, \mathbb{Z}_p)$, and the projective chain from the semidirect presentation to Paper-V1 presents $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. "Exact" is a central extension by $\langle \omega \rangle$, and the final two stages also include the direct factor $\langle -1 \rangle$.

The Route. We mostly follow the paper. Its normal boxes, its symplectic normal form, and its parametrized *box relations* reappear below. A box relation says that pushing a gate through a normal box yields an updated box and residual *dirty* gates. We also keep the reduction to eighteen relations of its Theorem 4.4, a step the paper already delegated to Agda. The one major divergence is the step that promotes symplectic to projective completeness, its Proposition 4.9. There, the paper corrects each rule $C_l = C_r$ to $C_l = C_r \,;\, P$ for the unique Pauli P that makes it projectively sound. It then replays the symplectic rewrites, pushing each correction to the end, where it becomes "a potentially different Pauli". We did not see how to verify this argument at reasonable cost. Instead, we work in a syntax where Paulis are trivial, since symplectic circuits have only H, S, and CZ and denote $\mathrm{Sp}(2n, \mathbb{Z}_p)$ itself. We then reintroduce Paulis and scalars by construction (Figure 16). The Pauli bookkeeping of Proposition 4.9 is thus done once, by a theorem about semidirect products that the framework proves for arbitrary groups.

D.5 Normal Forms for the Symplectic Factor

We now give the datatypes and proof obligations behind Figure 15. We model $\mathrm{Sp}(2n, \mathbb{Z}_p)$ as the group of $\mathbb{Z}_p$-linear bijections of the phase space $\mathsf{Pauli}\ n = \mathsf{Vec}\ (\mathbb{Z}_p \times \mathbb{Z}_p)\ n$ that preserve the symplectic form. An element of the phase space lists, for each wire, the exponents of $\mathrm{X}^a\mathrm{Z}^b$ up to phase. Against this semantics, completeness rests on the normal form, which we refine into a doubly inductive datatype:

```
M L ML : ℕ → Set
M 0 = ⊤
M (1+ n) = Vec D n × E
L 0 = ⊤
L (1+ n) = Vec B n × A
ML 0 = ⊤
ML 1 = M 1 × L 1
ML m@(2+ n) = M m × L m
            ⊎ D × ML (1+ n)
```

An M-row is a column of D boxes ending in an E box, the paper's X-normal layer. An L-row is a column of B boxes ending in an A box, its Z-normal layer, with the A box at the bottom. An ML box at width $m \geq 2$ is either a full pair of rows or a bottom D box followed by an ML box one wire up, so it is a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

```
NF : ℕ → Set
NF 0 = ⊤
NF (1+ n) = NF n × ML (1+ n)

[_] : ∀ {n} → NF n → Word (Gen n)
[_] {0}     tt       = ε
[_] {1+ n} (nf , ml) = [ nf ] ↑ • [ ml ]ml
```

The Table. The coset table is the paper's "push a gate through a normal box", written as a structurally recursive function ract with its soundness law ract-sound (about 5,600 lines together):

```
ract       : ML (1+ n) → Gen (1+ n) → Circuit n × ML (1+ n)
ract-sound : ∀ (c : ML (1+ n)) g → let (b' , c') = ract c g in [ c ]c • [ g ]w ≈ b' ↑ • [ c' ]c
```

Each clause of ract-sound is derived from the fifteen simplified symplectic relations. For instance, pushing S through an E box is one step on powers of S, namely $[\, b \,]^{\mathrm{e}} \bullet \mathrm{S} = \mathrm{S}^{-b} \bullet \mathrm{S} \approx \mathrm{S}^{-(b-1)} = [\, b-1 \,]^{\mathrm{e}}$. A circuit is rewritten to its normal form by repeating this step gate by gate, so every rewrite is traced to a relation.

Uniqueness. The symplectic group acts on Vec Pauli_1 n, a $2n$-dimensional symplectic $\mathbb{Z}_p$-vector space in which Pauli_1 is the two-dimensional symplectic subspace of a single wire. The elements that fix the bottom wire's subspace pointwise form a copy of $\mathrm{Sp}(2(n-1), \mathbb{Z}_p)$ acting above the bottom wire, and the ML boxes represent its cosets. Every $f \in \mathrm{Sp}(2n, \mathbb{Z}_p)$ factors as $f = f' \circ c_n$, with $c_n \in ML_n$ and f' acting above the bottom wire, and c_n is unique because the image of the bottom wire determines the box. Recursing on f' writes f uniquely as $c_1 \circ c_2 \circ \cdots \circ c_n$ with $c_k \in ML_k$, which is Theorem D.2.

Surjectivity. Every symplectic transformation is the action of some circuit. We prove this constructively by clearing the bottom wire and recursing on the rest. This is the paper's Lemma 3.7, with the coset box computed rather than asserted:

```
Theorem-LM : ∀ {n} (p q : Pauli n) → sform p q ≡ 1 → ∃ λ (lm : ML n) → act [ lm ]ml p ≡ pZ0 × act [ lm ]ml q ≡ pX0
```

Completeness. Circuits with the same denotation rewrite through the table to their normal forms. These normal forms denote the same map by soundness, so they are equal by Theorem D.2, and the circuits are equivalent. This argument is the framework's by-normalization (Figure 3a). With surjectivity, it shows that the simplified relations, which their own module also calls _SRel,_===_, present the symplectic group:

```
presentation : ∀ n → (n SRel._===_) IsPresentationOf (Sp-group n)
```

D.6 Restoring Paulis and the Source Rule Sets

$\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ is presented over X and Z on each wire by $X^p = Z^p = \varepsilon$ and ZX = XZ. Its semantic group is the phase space that $\mathrm{Sp}(2n, \mathbb{Z}_p)$ acts on, and its normal form is $X^a Z^b$ on each wire. Combining this with the symplectic presentation of Section D.5 is an instance of the framework's theorem for semidirect products. The resulting presentation has the relations of both factors together with the conjugation relations $h\, n\, h^{-1} = \mathit{conj}\, h\, n$, where the action conj spells out the paper's Lemmas 2.24–2.25. The theorem asks that the action respect the relations of each factor, and we prove this semantically. The conjugates of congruent words have equal denotations by soundness, so they are congruent by Pauli *completeness*. For symplectic rules this is respects-Δ, and it corresponds to the paper's "Pauli complete" rules (its Definition 4.7). Since every premise is a theorem, the semidirect presentation of Pauli⋊Sp is unconditional.

Reaching the Paper's Rules. The semidirect presentation has generators X, Z, H, S, and CZ and 3 + 15 + 14 relations (Pauli, symplectic, and conjugation), whereas the paper's rule sets have only H, S, and CZ. Three isomorphisms of presentations close this gap (Figure 16), through `Simplified-V1` (nineteen rules), `Paper-V0` (sixteen), and `Paper-V1` (the fifteen of Theorem D.1). Along the way, X and Z become words, and the Pauli and conjugation rules become theorems.

The first isomorphism, from the semidirect presentation to `Simplified-V1`, is between different alphabets. Its map f sends S to $R = \mathrm{S} \bullet \mathrm{Z}^{1/2}$, and its inverse h sends S to $\mathrm{Z}^{-1/2} \bullet \mathrm{S}$. In particular, f induces the embedding of the symplectic group into the projective Clifford group. The nineteen rules of `Simplified-V1` are the fifteen simplified symplectic rules together with $(\mathrm{SH})^3 = \varepsilon$, $\mathrm{H}^2\mathrm{SH}^2\mathrm{S} = \mathrm{SH}^2\mathrm{SH}^2$, and the two rules pushing X through CZ. From these, f-well-defined derives the images under f of the three Pauli rules and the fourteen conjugation rules.

The second isomorphism, from `Simplified-V1` to `Paper-V0`, is the identity on circuits. The two rule sets share six rules word for word and three more up to the spelling of the multiplier, and each derives the rest of the other. The larger direction is `Paper-V0`'s module `Lemmas`, of about 4,750 lines.

The third isomorphism, from `Paper-V0` to `Paper-V1`, changes how the multiplier $M_x : |j\rangle \mapsto |x\,j\rangle$ is spelled as a word. `Paper-V0` spells it over R, as $M_x = R^{x^{-1}}\mathrm{H}R^{x}\mathrm{H}R^{x^{-1}}\mathrm{H}$. The paper's (T1) spells it over S, up to a phase, as $M_x = \mathrm{SHS'}\ x\ x^{-1}$, where $\mathrm{SHS'}\ a\ b = \mathrm{Z}^{(b-1)/2}\mathrm{X}^{(1-a)/2}\mathrm{S}^b\mathrm{HS}^a\mathrm{HS}^b\mathrm{H}$. Over S, the rule semi-MR reads $M_g \mathrm{S}^{g^2} \mathrm{Z}^{(g^2-g)/2} = \mathrm{S}M_g$. `Paper-V1` restates the four multiplier rules over SHS' and drops one rule. Since M_1 is now the word $(\mathrm{SH})^3$, M-power at $k = 0$ already gives $(\mathrm{SH})^3 \approx \varepsilon$. Over R, M_1 is the different word $(R\mathrm{H})^3$, so that axiom had to stay. The bridge $R^a\mathrm{H}R^b\mathrm{H}R^a\mathrm{H} \approx \mathrm{SHS'}\ b\ a$ for $ab = 1$ (`Shared.PauliBase`) carries the multiplier rules across, which gives Theorem D.1. The paper's own rule (C4) states the conjugation of S by M_g in the other direction. Each of (C4) and semi-MR derives the other (SemiMS), so Figure 1 of Bian, Li et al. [12] read modulo scalars is isomorphic to `Paper-V1`.

D.7 Restoring Scalar Phases

The theorem extension-presentation extends a projective presentation by a scalar. We apply it to `Paper-V0`, which has order-SH as an axiom. We add ω with $\omega^p = \varepsilon$, let ω commute with every gate, and *correct* each rule by its scalar. The correction is ε except on order-SH, the one rule that does not hold exactly, since $(\mathrm{SH})^3 = \omega^{-1/8}$. The exponent $-1/8$ is computed in $\mathbb{Z}_p$, where it equals $(p^2-1)/8$. The multiplier words now denote $|j\rangle \mapsto |x\,j\rangle$ only up to a scalar.

The semantic extension is the Weyl cocycle on the natural semidirect product, $c((P,A),(Q,B)) = \frac{1}{2}\,\mathrm{sform}(P,\ A\cdot Q)$. Here $A\cdot Q$ is the symplectic part acting on the Pauli vector, and sform is the symplectic form of the phase space, the exponent in $PQ = \omega^{\mathrm{sform}(P,Q)}\,QP$. The cocycle identity follows from the bilinearity and the symplectic invariance of sform. The cocycle of the presented group is the pullback of c along the presentation by `Paper-V0`, so the two agree by construction. The identification with unitary operators uses Weyl multiplication [39, Eq. (4), Theorem 3] and a *linear* Weil representation with exact covariance [40, Theorem 1.3.2]. The operators $U(e,v,A) = \omega^e W(v)\rho(A)$ satisfy

$$U(e,v,A)\,U(f,w,B) = U\big(e+f+\tfrac{1}{2}\mathrm{sform}(v,Aw),\ v+Aw,\ AB\big).$$

This is the Heisenberg–Weil representation [41, §1.3]. Conjugation on Weyl operators determines A and v, so the representation is faithful and its scalars are exactly $\langle\omega I\rangle$ (at width zero, $e\mapsto\omega^e$). Adjoining $-I$ gives its direct product with $\mathbb{Z}_2$, whose scalars are $\langle -\omega I\rangle$. These finite phase choices differ from the full unitary normalizer, which contains $U(1)$. With no hypotheses, we obtain:

```
presentation-exact : ∀ n → (n Exact,_===_) IsPresentationOf (PE.Presented-group n (exact-generator-data n))
```

Agda also shows that the trivial cocycle cannot realize the correction of order-SH (split-forces).

The Rules as Printed. Figure 1 of Bian, Li et al. [12] is stated over $-\omega$. Its multipliers carry Legendre symbols and its swap carries the sign $(-1)^{(p-1)/2}$, so it needs the -1 that the exact group lacks, since the scalars of that group have odd order. Adjoining -1 as a central involution presents the exact group times $\mathbb{Z}_2$, which is Clifford±. Figure 1 is isomorphic to this rule set by $-\omega\mapsto -1\cdot\omega$, where the scalar of each exact rule is determined by reading its lift back. This is the paper's Theorem 4.10, stated against our group:

```
presentation-Figure1 : ∀ n → (n F,_===_) IsPresentationOf (Clifford± n)
```

D.8 Implementation and Semantic Boundary

This development is checked by Agda 2.8 with standard library 2.4 under `--safe`, with no postulates and no holes. The development has 215 modules and about 58,000 lines of code, of which about 27,000 are for the symplectic group, 18,000 for the projective chain, and 13,000 for the scalars. These counts exclude the shared library (Appendix C).

The Agda development does not formalize the two isomorphisms of Section D.4 [39, 40], and no matrix appears in it. The representation argument above is external mathematics, and matching the paper's gate phases remains a convention check [12, 56]. The Lean development needs neither, since it proves soundness and completeness against complex matrices directly (Appendix C).

We used versions of Claude for parts of this formalization: composing box relations to prove ract-sound, defining the symplectic group, and establishing the equivalence between the code's relations and those in Figure 1 of Bian, Li et al. [12], read modulo scalars. Claude also wrote presentation-exact and the layer for Figure 1 with -1.

E Proofs for Four-Bit Reversible Circuits

This appendix proves Theorems 6.1 and 6.2 as the Agda development [5] does. Each subsection names the modules of `Examples/Groups/Alt16-RevCir` that it follows, and each statement names its Agda counterpart.

E.1 Conventions and Outline

The gates and the theory are the modules Syntactics and Theory, and the proof engine is Engine. Wires are numbered from 0 at the bottom. A state of k wires is a bit string $b_{k-1}\cdots b_0$, the bit of the

top wire first. Circuits are words in diagram order, and products of permutations follow the same order, so uw applies u and then w. We write $u \approx w$ when u and w are equal modulo $\mathcal{T}$, ε for the empty circuit, and G_k for the circuits on k wires modulo $\mathcal{T}$, with the product of circuits.

The gate X acts on wire 0, CX has its control on wire 1 and its target on wire 0, CCX has its controls on wires 2 and 1 and its target on wire 0, and the structural swap σ exchanges the wires 0 and 1. The *generators* of the circuits on k wires are these four gates, shifted up by any number of wires. A gate in any other position is a generator between swaps, and an open control is a filled one between two X gates on its wire. We write X_t, $\mathrm{CX}_{c\to t}$, and $\mathrm{CCX}_{ab\to t}$ for the gates with target t and controls c, or a and b in either order, with a bar over an open control as in $\mathrm{CCX}_{\bar{3}2\to 0}$, and σ_d for the swap of the wires d and $d+1$. On four wires there are twelve generators: $\mathrm{X}_0, \ldots, \mathrm{X}_3$, $\mathrm{CX}_{1\to 0}$, $\mathrm{CX}_{2\to 1}$, $\mathrm{CX}_{3\to 2}$, $\mathrm{CCX}_{21\to 0}$, $\mathrm{CCX}_{32\to 1}$, and σ_0, σ_1, σ_2. In this notation the equations of Figure 9 are

$$\text{(T1)}\ \mathrm{X}_0\,\mathrm{X}_0 = \varepsilon, \qquad \text{(T2)}\ \mathrm{CX}_{1\to 0}\,\mathrm{X}_1\,\mathrm{CX}_{1\to 0}\,\mathrm{X}_1 = \mathrm{X}_0,$$
$$\text{(T3)}\ \mathrm{CX}_{1\to 0}\,\mathrm{CX}_{0\to 1}\,\mathrm{CX}_{1\to 0} = \sigma_0, \qquad \text{(T4)}\ \mathrm{CCX}_{21\to 0}\,\mathrm{X}_2\,\mathrm{CCX}_{21\to 0}\,\mathrm{X}_2 = \mathrm{CX}_{1\to 0},$$
$$\text{(T5)}\ \sigma_1\,\mathrm{CCX}_{21\to 0}\,\sigma_1 = \mathrm{CCX}_{21\to 0},$$
$$\text{(T6)}\ \mathrm{CX}_{0\to 1}\,\mathrm{CCX}_{21\to 0}\,\mathrm{CX}_{0\to 1} = \mathrm{CCX}_{20\to 1}\,\mathrm{CCX}_{21\to 0}\,\mathrm{CCX}_{20\to 1},$$
$$\text{(T7)}\ \mathrm{CCX}_{32\to 1}\,\mathrm{CX}_{1\to 0}\,\mathrm{CCX}_{32\to 1} = \mathrm{CCX}_{32\to 0}\,\mathrm{CX}_{1\to 0},$$

where $\mathrm{CX}_{0\to 1} = \sigma_0\,\mathrm{CX}_{1\to 0}\,\sigma_0$, $\mathrm{CCX}_{20\to 1} = \sigma_0\,\mathrm{CCX}_{21\to 0}\,\sigma_0$, and $\mathrm{CCX}_{32\to 0} = \sigma_0\,\mathrm{CCX}_{32\to 1}\,\sigma_0$.

Derivations are checked by a small proof engine. An equation is a pair of lists of generators with a derivation between their circuits. A new equation is derived by a list of steps, each of which rewrites an occurrence of one side of a known equation, at a given place in the list, into the other side, and Agda checks the derivation by running the steps. The derivations are generated by scripts that search for them and that replay every step before they write it. A fact is *decided by evaluation* when it is stated as a Boolean that Agda's type checker computes to be true.

The step between two presentations. The proof uses the following form of the Reidemeister–Schreier method several times, as StarGroupIsomorphism of the framework's Presentation.Morphism. Let $\langle A \mid R\rangle$ and $\langle B \mid Q\rangle$ be two presentations, and let f send each generator in A to a word over B and g each generator in B to a word over A. Both extend to words, letter by letter. Suppose that

(1) for every relation $l = r$ of R, the words $f(l)$ and $f(r)$ are equal modulo Q;
(2) for every relation $l = r$ of Q, the words $g(l)$ and $g(r)$ are equal modulo R;
(3) every generator a in A is equal to $g(f(a))$ modulo R;
(4) every generator b in B is equal to $f(g(b))$ modulo Q.

If moreover every generator has an inverse word on each side, then f induces an isomorphism from the group of $\langle A \mid R\rangle$ onto that of $\langle B \mid Q\rangle$, with the inverse induced by g. We call (1)–(4) the *four facts* of the step.

Outline. The proof has five parts.

(1) Lafont's presentation of Alt(16) is carried to the four-wire circuits by a first step. Its relations on the circuits form the *translated relation*, which is therefore complete (Appendices E.2–E.4).
(2) On at most three wires, $\mathcal{T}$ is complete, by a step from the Coxeter presentation of the symmetric group (Appendix E.5).
(3) A four-wire derivation is built from circuits on three adjacent wires, which the second part decides, and from thirteen instances of the rules on all four wires. Two lemmas about circuits of X gates and swaps make it possible to move a gate to any wires (Appendix E.6).

(4) The equations of the translated relation are derived from $\mathcal{T}$: the relations between the cycles, by derivations that a search finds, and then the six equations for the gates (Appendices E.7–E.9).
(5) A second step identifies the two congruences on four wires, which gives Theorem 6.1, and Lafont's canonical forms give Theorem 6.2 (Appendices E.10 and E.11).

No derivation is written by hand except for the short ones behind the two lemmas of the third part. The first and the last part rest on semantics and on Lafont's theorem, and the derivations of the second and the fourth are found by search.

E.2 Soundness

This part follows the modules Semantics and Theory.

Proposition E.1 (T-sound). *On any number of wires, if $u \approx w$ then $[\![u]\!] = [\![w]\!]$.*

Proof. The two sides of each of (T1)–(T7) denote the same function at the width at which the equation is drawn, which is decided by evaluation on all states of that width. The structural rules hold at every width, by computation on the bits of the wires that they touch. The claim follows by induction on the derivation. □

E.3 Lafont's Presentation and the Cycles

This part follows the modules Numbering and Syntactics, and the library's Alternating. As in Section 6, $\omega_0, \ldots, \omega_{N-3}$ are the cycles of three consecutive points among N points. Lafont's relations between these cycles are

$$\begin{array}{ll} \text{(order)}\ \ \omega_i\,\omega_i\,\omega_i = 1, & \text{(near)}\ \ \omega_i\,\omega_{i+1}\,\omega_i = \omega_{i+1}\,\omega_{i+1}, \\ \text{(far)}\ \ \omega_i\,\omega_{i+2}\,\omega_i = \omega_{i+2}\,\omega_i\,\omega_{i+1}, & \text{(apart)}\ \ \omega_j\,\omega_i = \omega_i\,\omega_j \quad (j \geq i+3), \end{array}$$

with products in diagram order. The first three are the relations of his Theorem 4 as he draws them, and (apart) is the commutation rule of his diagrams. In formulas he writes products in the order of composition, so that the right side of (far) reads $\omega_{i+1}\,\omega_i\,\omega_{i+2}$ there, and he replaces (near) by $(\omega_i\,\omega_{i+1})^2 = 1$, which is equivalent to it given (order). The library of EquaCert proves Theorem 4 of [47]: two words in the ω_i with the same permutation are equal modulo these relations, and every even permutation of the N points is that of a word. In the library the ω_i are one three-wire gate shifted up, and (apart) is a structural rule. For $N = 16$ there are 14 cycles and $14 + 13 + 12 + 66 = 105$ relations.

Positions. The sixteen states of four wires are numbered along the reflected Gray code, in which wire 0 changes most often. Position i is the state $b_3b_2b_1b_0$ with the binary digits of $i \oplus \lfloor i/2 \rfloor$:

0	0000	4	0110	8	1100	12	1010
1	0001	5	0111	9	1101	13	1011
2	0011	6	0101	10	1111	14	1001
3	0010	7	0100	11	1110	15	1000

Two consecutive positions differ on one wire. So the positions i, $i+1$, $i+2$ are three corners of a face of the 4-cube, on which two wires change and the other two hold fixed values.

The cycles. Let $K = (\sigma_0\,\sigma_2\,\mathrm{CCX}_{21\to0})^4$. Between $\sigma_0\sigma_2$ and itself the Toffoli gate $C = \mathrm{CCX}_{21\to0}$ is $T = \mathrm{CCX}_{30\to1}$, so K has the function of $TCTC$, the commutator of two Toffoli gates (Figure 12(a)). When the wires 2 and 3 are both 1 it cycles the states 01, 11, 10 of the wires 1 and 0, and it fixes every other state: it is the cycle of the positions 9, 10, 11. Swaps carry it to the other faces. Let $K_{10} = K$, $K_{01} = \sigma_0 K \sigma_0$, $K_{20} = \sigma_1 K \sigma_1$, $K_{02} = \sigma_1 K_{01} \sigma_1$, $K_{30} = \sigma_2 K_{20} \sigma_2$, and $K_{03} = \sigma_2 K_{02} \sigma_2$. Then K_{ab}

cycles x, $x + e_a$, $x + e_a + e_b$, where x is the state in which wire a is 0 and the other wires are 1, and e_a flips wire a. If wire a changes from position i to $i + 1$ and wire b from $i + 1$ to $i + 2$, the cycle Ω_i is K_{ab} between X gates on the wires on which position i differs from x. For example, $\Omega_9 = K$, $\Omega_4 = \mathrm{X}_3 K_{01} \mathrm{X}_3$, and $\Omega_0 = \mathrm{X}_1\mathrm{X}_2\mathrm{X}_3\, K_{01}\, \mathrm{X}_3\mathrm{X}_2\mathrm{X}_1$. Written out, Ω_i has between 12 and 24 generators.

Lemma E.2 (Bridge.g-sem-gen). *For $0 \le i \le 13$, the circuit Ω_i carries position i to $i + 1$, $i + 1$ to $i + 2$, and $i + 2$ to i, and fixes the other positions.*

Proof. This is decided by evaluation at the sixteen positions, for each of the fourteen circuits. □

E.4 The First Step: the Translated Relation

This part follows the modules Translation, Lemmas, Bridge, Iso, and Presentation. The step is between the four-wire circuits and Lafont's words for $N = 16$.

The translations. The translation g reads a cycle as a circuit, $g(\omega_i) = \Omega_i$. The translation f reads a generator as a word in the cycles that carries out, on the positions, the permutation of the gate. On the gates on their own wires,

$$f(\mathrm{X}_0) = \omega_1\omega_0\,\omega_5\omega_4\,\omega_9\omega_8\,\omega_{13}\omega_{12}, \qquad f(\mathrm{CX}_{1\to0}) = \omega_3\omega_2\,\omega_{11}\omega_{10},$$
$$f(\mathrm{CCX}_{21\to0}) = \omega_5\omega_4\,\omega_7\omega_6\,\omega_9\omega_8, \qquad f(\sigma_0) = \omega_1\,\omega_3\omega_2\,\omega_4\,\omega_9\,\omega_{11}\omega_{10}\,\omega_{12},$$

and $f(\sigma_1)$ and $f(\sigma_2)$ are words of 12 and 24 cycles. Here $\omega_{i+1}\omega_i$ exchanges the positions i and $i + 1$ and the positions $i + 2$ and $i + 3$, and for instance X_0 exchanges the positions $2m$ and $2m + 1$ for every m. A generator further up goes to the word of the same gate one wire lower, between the words of the swaps that carry it there: $f(\mathrm{X}_{t+1}) = f(\sigma_t)\, f(\mathrm{X}_t)\, f(\sigma_t)$, and similarly for $\mathrm{CX}_{2\to1}$, $\mathrm{CX}_{3\to2}$, and $\mathrm{CCX}_{32\to1}$.

The translated relation. It is a relation on the four-wire circuits, with the structural rules of Figure 10 and two kinds of equations. The first are Lafont's relations read through g, that is (order), (near), (far), and (apart) with Ω in place of ω, in their 105 instances. The second are six *gate equations*, which say that each gate on its own wires is the circuit of its word: $\mathrm{X}_0 = g(f(\mathrm{X}_0))$, and likewise for $\mathrm{CX}_{1\to0}$, $\mathrm{CCX}_{21\to0}$, σ_0, σ_1, and σ_2. We write $u \approx_L w$ for equality modulo the translated relation.

Lemma E.3 (Bridge.f-sem, Bridge.g-sem). *For a four-wire circuit u, the word $f(u)$ permutes the positions as u maps the states. For a word v in the cycles, the circuit $g(v)$ maps the states as v permutes the positions.*

Proof. For a generator and for a cycle this is decided by evaluation at the sixteen positions; for the cycles it is Lemma E.2. The claims follow by induction on the word. Every statement about permutations is made position by position. □

Proposition E.4 (Iso.Theorem, Presentation). *The four facts hold for f and g between the translated relation and Lafont's relations. Hence the four-wire circuits modulo the translated relation form a group that $u \mapsto f(u)$ maps isomorphically onto Lafont's, and so onto* Alt(16) (presentation). *In particular, two four-wire circuits with the same function are equal modulo the translated relation* (complete)*, and conversely* (sound).

Proof. The two facts toward the circuits are syntactic. An instance of one of Lafont's relations, read through g, is one of the 105 equations, by recursion on the instance with the number of shifts as a parameter (lemma-g-rel). A gate on its own wires is the circuit of its word by its gate equation. A generator further up is that gate between swaps, by the naturality of the swap, for example $\mathrm{X}_1 \approx_L \sigma_0\mathrm{X}_0\sigma_0$ and $\mathrm{CCX}_{32\to1} \approx_L \sigma_0\sigma_1\sigma_2\,\mathrm{CCX}_{21\to0}\,\sigma_2\sigma_1\sigma_0$, and f was defined to follow (lemma-g-f).

The two facts toward Lafont's words are semantic. Each equation of the translated relation holds for functions on states, which is decided by evaluation on the sixteen states; the equations with indices are decided for all their indices by one Boolean each. So by Lemma E.3 the words of its two sides have the same permutation, and they are equal modulo Lafont's relations because his presentation is complete. In the same way, ω_i and $f(g(\omega_i)) = f(\Omega_i)$ have the same permutation. Hence no derivation between words in the cycles is written out.

Lafont's generators have inverse words, and the inverse of a generator x of the circuits is the circuit of the inverse of $f(x)$. The step gives the isomorphism, and Lafont's theorem identifies his group with Alt(16). Completeness follows because the isomorphism is injective and, by Lemma E.3, two circuits with the same function have words with the same permutation. □

E.5 At Most Three Wires

This part follows the modules BySymmetric, Width1, Width2, Width3, and ThreeWire. The library of EquaCert proves the Coxeter presentation of $\mathrm{Sym}(N)$ [29]. Its generators are the transpositions τ_d of d and $d+1$, for $0 \le d \le N-2$, and its relations are $\tau_d\tau_d = 1$, $\tau_d\tau_{d+1}\tau_d = \tau_{d+1}\tau_d\tau_{d+1}$, and $\tau_e\tau_d = \tau_d\tau_e$ for $e \ge d+2$. Two words in the τ_d with the same permutation are equal modulo these relations, and every permutation is that of a word.

On $k \le 3$ wires, number the $N = 2^k$ states along the reflected Gray code, as in Appendix E.3. The states numbered d and $d+1$ differ on one wire, and the *edge gate* Θ_d is the X gate on that wire, controlled by every other wire, each control filled or open according to the bit that the two states share on its wire. It exchanges the two states and fixes the others. On one wire, $\Theta_0 = \mathrm{X}_0$. On two wires, $\Theta_0 = \mathrm{CX}_{\bar{1}\to 0}$, $\Theta_1 = \mathrm{CX}_{0\to 1}$, and $\Theta_2 = \mathrm{CX}_{1\to 0}$. On three wires the seven edge gates are Toffoli gates,

$$\Theta_0 = \mathrm{CCX}_{\bar{2}\bar{1}\to 0}, \quad \Theta_1 = \mathrm{CCX}_{\bar{2}0\to 1}, \quad \Theta_2 = \mathrm{CCX}_{\bar{2}1\to 0}, \quad \Theta_3 = \mathrm{CCX}_{1\bar{0}\to 2},$$
$$\Theta_4 = \mathrm{CCX}_{21\to 0}, \quad \Theta_5 = \mathrm{CCX}_{20\to 1}, \quad \Theta_6 = \mathrm{CCX}_{2\bar{1}\to 0},$$

each written as $\mathrm{CCX}_{21\to 0}$ between X gates on its open controls and the swaps that bring its wires into place.

Proposition E.5 (complete$_1$–complete$_3$, presentation$_1$–presentation$_3$). *For $1 \le k \le 3$, the theory $\mathcal{T}$ is complete on k wires, G_k is a group, and $u \mapsto [\![u]\!]$ is an isomorphism from G_k onto $\mathrm{Sym}(2^k)$.*

Proof. The module BySymmetric carries out the step between the circuits on k wires modulo $\mathcal{T}$ and the Coxeter presentation of $\mathrm{Sym}(2^k)$, for any k, from the two facts toward the circuits and from the semantics of the two translations on generators. The translation g reads τ_d as Θ_d, and f reads a generator as a word in the transpositions. On three wires, for example,

$$f(\mathrm{X}_0) = \tau_0\tau_2\tau_4\tau_6, \qquad f(\mathrm{CX}_{1\to 0}) = \tau_2\tau_4, \qquad f(\mathrm{CCX}_{21\to 0}) = \tau_4, \qquad f(\sigma_0) = \tau_1\tau_2\tau_1\,\tau_4\tau_5\tau_4,$$

$f(\sigma_1)$ has 14 letters, and the generators further up are between the words of swaps, as in Appendix E.4.

The two facts toward the Coxeter words are semantic, as in Proposition E.4: $\mathcal{T}$ is sound by Proposition E.1, a word in the transpositions permutes the numbers as its circuit maps the states, which is decided by evaluation on the generators, and the Coxeter presentation is complete. The two facts toward the circuits are derivations from $\mathcal{T}$. They ask for the Coxeter relations between the Θ_d, and for each gate on its own wires to be the circuit of its word; a generator further up then follows by the naturality of the swap. On one wire the only relation is (T1). On two wires these are nine equations, and on three wires 33: seven of the form $\Theta_d\Theta_d \approx \varepsilon$, six braid relations, fifteen commutations, and five gates.

A Knuth–Bendix completion [46] finds the derivations. It starts from the instances of the rules on the given wires, oriented as rewrite rules by length and then alphabetically: seven instances

on two wires, and 23 on three, of (T1)–(T6), of the laws of the swap, and of the commutations of generators on disjoint wires. Each rule that the completion adds comes from an overlap of two earlier rules, and is thereby an equation derived from them. The completion stops as soon as the two sides of every target rewrite to the same word. The derivations of the targets use 16 added rules on two wires and 316 on three. Each added rule and each target becomes one derivation of the engine, 1,596 steps in all. On three wires only (T1)–(T6) are used, since (T7) needs four.

The step makes f an isomorphism from G_k onto the group of the Coxeter presentation, hence onto $\mathrm{Sym}(2^k)$, and completeness follows as in Proposition E.4. □

E.6 The Steps of a Four-Wire Derivation

This part follows the modules FourWire, Frames, and Letters. Each of the twelve generators on four wires lies on the bottom three wires, on the top three, or on both.

Lemma E.6 (three-wire steps, bot, top). *Two circuits on the wires 0, 1, 2 of four wires that denote the same function are equal modulo $\mathcal{T}$, and so are two circuits on the wires 1, 2, 3.*

Proof. As circuits on three wires they are equal by Proposition E.5. The structural rules carry the derivation to four wires, with an idle wire on top or, by (shift), below. That the two functions agree is decided by evaluation on the eight states. □

Thirteen instances of the rules involve all four wires and are not three-wire steps. They are (T7); the naturality of the swap at the Toffoli gate, $\mathrm{CCX}_{21\to 0}\,\sigma_2\sigma_1\sigma_0 \approx \sigma_2\sigma_1\sigma_0\,\mathrm{CCX}_{32\to 1}$; and eleven commutations of a generator that touches wire 0 with a generator that touches wire 3 and none of its wires: X_0 with X_3, $\mathrm{CX}_{3\to 2}$, σ_2, and $\mathrm{CCX}_{32\to 1}$; each of $\mathrm{CX}_{1\to 0}$ and σ_0 with X_3, $\mathrm{CX}_{3\to 2}$, and σ_2; and $\mathrm{CCX}_{21\to 0}$ with X_3. Every four-wire derivation of the development is built from three-wire steps and these thirteen equations.

Symmetry words. A *symmetry word* is a word in $\mathrm{X}_0, \ldots, \mathrm{X}_3$ and $\sigma_0, \sigma_1, \sigma_2$. These circuits carry out the 384 symmetries of the 4-cube, which permute the wires and complement some of them. The reversed word p^{-1} of a symmetry word p has the inverse function. A *normal form* is a triple (q, e, r) of a symmetry word q on the bottom three wires, a bit e, and one of the four words $r \in \{\varepsilon,\ \sigma_2,\ \sigma_2\sigma_1,\ \sigma_2\sigma_1\sigma_0\}$, which bring each of the four wires to the top. Its circuit is $q\,\mathrm{X}_3^e\,r$.

Lemma E.7 (hstep). *Every symmetry word p has a normal form $\mathrm{nf}(p) = (q, e, r)$ with $p \approx q\,\mathrm{X}_3^e\,r$. Hence two symmetry words are equal modulo $\mathcal{T}$ if their normal forms have the same e, the same r, and words q with the same function.*

Proof. The normal form is computed one letter at a time, from $(\varepsilon, 0, \varepsilon)$. A table gives, for each of the four words r and each of the seven letters y, a letter or the empty word b on the bottom three wires, a bit e', and a word r' with $r\,y \approx b\,\mathrm{X}_3^{e'}\,r'$. For example, $\sigma_2\sigma_1\,\mathrm{X}_1 \approx \mathrm{X}_3\,\sigma_2\sigma_1$ and $\sigma_2\sigma_1\sigma_0\,\sigma_1 \approx \sigma_0\,\sigma_2\sigma_1\sigma_0$. Each of the 28 entries is derived in at most three steps, which are three-wire steps and commutations. Since X_3 commutes with every generator on the bottom three wires, by the structural rules, appending y to $q\,\mathrm{X}_3^e\,r$ gives $(qb)\,\mathrm{X}_3^{e+e'}\,r'$, which proves the first claim by induction. For the second, the two words q are equal by a three-wire step. In the development the comparison of two normal forms is a Boolean, which Agda evaluates for each pair of words to which the lemma is applied. □

Frames. For a symmetry word p and a circuit b on the bottom three wires, the circuit $p\,b\,p^{-1}$ is b *in the frame* p. It acts as b on the wires that p brings to the bottom, with the polarities that p sets.

Lemma E.8 (change of frame, frame). *Let p and p' be symmetry words and b and b' circuits on the bottom three wires. Suppose that* $\mathrm{nf}(p^{-1}p') = (q, e, \varepsilon)$ *and* $\mathrm{nf}(p'^{-1}p) = (\bar{q}, e, \varepsilon)$*, and that $q\,b'\,\bar{q}$ has the function of b. Then $p\,b\,p^{-1} \approx p'\,b'\,p'^{-1}$.*

Proof. By Lemma E.7, $p\,p^{-1} \approx \varepsilon$, so $p' \approx p\,(p^{-1}p') \approx p\,q\,\mathrm{X}_3^e$ and $p'^{-1} \approx (p'^{-1}p)\,p^{-1} \approx \bar{q}\,\mathrm{X}_3^e\,p^{-1}$. The gate X_3 commutes with b' and with $\bar{q}$, and $\mathrm{X}_3\mathrm{X}_3 \approx \varepsilon$, so

$$p'\,b'\,p'^{-1} \approx p\,q\,\mathrm{X}_3^e\,b'\,\bar{q}\,\mathrm{X}_3^e\,p^{-1} \approx p\,(q\,b'\,\bar{q})\,p^{-1} \approx p\,b\,p^{-1},$$

the last by a three-wire step. The hypotheses are one Boolean, evaluated for each use. □

The hypothesis on the normal forms says that p and p' bring the same wire to the top. The two lemmas are the only general facts about four wires that the development proves by hand. In the generated derivations a step is a three-wire step, one of the thirteen equations, an instance of Lemma E.7 or of Lemma E.8, or an equation derived before.

E.7 Placed Gates and Pair Lemmas

This part follows the generated modules Four.Part1–Four.Part14 and the scripts that write them. The search works with the 76 *placed gates* on four wires: four X_t, the 24 gates $\mathrm{CX}_{c\to t}$ with a filled or an open control, and the 48 gates $\mathrm{CCX}_{ab\to t}$ with each control filled or open. As a circuit, X_t is a generator, and every other placed gate is the base gate $\mathrm{CX}_{1\to 0}$ or $\mathrm{CCX}_{21\to 0}$ in a frame: X gates on its open controls, followed by the swaps that bring its target and its controls to the wires 0, 1, and 2. By Lemma E.8, a placed gate conjugated by a symmetry word is the placed gate that the symmetry carries it to, whatever the frames.

A *macro derivation* rewrites a word of placed gates, in which gates on disjoint wires may be interchanged, by three kinds of move. A *narrow* move replaces consecutive gates on three of the four wires by gates on the same wires with the same function. The other two use a *pair lemma*: an equation $(g\,h)^2 = k$ for two placed gates g and h that together touch all four wires, where k is the identity or a placed gate. A *pair* move rewrites $g\,h$ to $k\,h\,g$ or to $h\,g\,k$, and an *expand* move rewrites k to $g\,h\,g\,h$. Each move becomes a few steps of the engine.

- *Narrow.* The frame of the wires 0, 1, 2 is empty, and those of the triples 0, 1, 3, then 0, 2, 3, then 1, 2, 3 are σ_2, $\sigma_1\sigma_2$, and $\sigma_0\sigma_1\sigma_2$. Each gate of the block is brought to the frame of the triple by Lemma E.8, the frames between consecutive gates cancel by Lemma E.7, a three-wire step on the bottom wires replaces the block, and the new gates are brought back. A CNOT has two idle wires, and when its frame brings the wrong one to the top, the frame is first extended by σ_2, which commutes with $\mathrm{CX}_{1\to 0}$.
- *Interchange.* An X gate passes a placed gate that does not touch its wire: it moves through the frame by Lemma E.7, where it becomes X_2 or X_3, passes the base gate by a three-wire step or a commutation, and moves back. Two CNOTs on complementary pairs of wires are $\mathrm{CX}_{1\to 0}$ and $\mathrm{CX}_{3\to 2}$ in a common frame, where they commute by a structural rule.
- *Pair and expand.* From $(g\,h)^2 \approx k$ and the involutions $g\,g \approx h\,h \approx k\,k \approx \varepsilon$, which are three-wire steps inside frames, one gets $g\,h \approx k\,h\,g$ and $g\,h \approx h\,g\,k$ by inserting $g\,g$ and $h\,h$.

There are 2,496 ordered pairs (g, h) with a pair lemma, and the symmetries of the cube permute them in 12 orbits. Table 3 lists a representative of each.

Proposition E.9 (pair lemmas). *The representatives of the twelve orbits hold modulo $\mathcal{T}$, and so does every pair lemma that a derivation of the development uses.*

Proof. *Equation* (T7). Let $T = \mathrm{CCX}_{32\to 1}$, $c = \mathrm{CX}_{1\to 0}$, and $Q = \mathrm{CCX}_{32\to 0}$. The naturality of the swap at the Toffoli gate and Lemma E.7 identify the generator T with $\mathrm{CCX}_{21\to 0}$ in the frame

Table 3. The twelve orbits of pair lemmas in the order in which they are derived, down the left column and then the right, with the number of moves of the macro derivation of the representative. The first is (T7).

$g,\ h$	$(g\,h)^2$	moves	$g,\ h$	$(g\,h)^2$	moves
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{32\to1}$	$\mathrm{CCX}_{32\to0}$	–	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{32\to1}$	$\mathrm{CCX}_{32\to0}$	21
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{32\to0}$	identity	7	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{3\bar{2}\to1}$	identity	12
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{31\to2}$	identity	6	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{21\to3}$	identity	6
$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{3\bar{1}\to2}$	identity	7	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{\bar{2}1\to3}$	identity	6
$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{31\to0}$	identity	54	$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{\bar{2}\bar{1}\to3}$	identity	9
$\mathrm{CCX}_{21\to0},\ \mathrm{CCX}_{3\bar{1}\to0}$	identity	8	$\mathrm{CX}_{1\to0},\ \mathrm{CCX}_{30\to2}$	$\mathrm{CCX}_{31\to2}$	7

$\sigma_0\sigma_1\sigma_2$, and $\sigma_0 T\sigma_0$ with the placed gate Q. Then (T7), which reads $TcT \approx Qc$, gives $(T\,c)^2 \approx Q$ after multiplying by c, an involution.

The other order. If $(g\,h)^2 \approx k$, then $(h\,g)^2 \approx k$: since $k\,k \approx \varepsilon$, we have $hghg \approx hghg\,(ghgh)\,k \approx k$, where the middle cancels pair by pair.

Transport. If $(g\,h)^2 \approx k$ and w is a symmetry word, then each of $w\,g\,w^{-1}$, $w\,h\,w^{-1}$, and $w\,k\,w^{-1}$ is the placed gate that the symmetry gives, by Lemma E.8, and the frames between them cancel. So the lemma holds for the pair that w carries (g, h) to. The development derives in this way each instance that a derivation uses, from the representative of its orbit.

The representatives. For each orbit after the first, a search finds a macro derivation that reduces the word $g\,h\,g\,h\,k$ of the representative to the empty word, with pair and expand moves only for the orbits derived before. Table 3 gives the order and the sizes. The second orbit, for example, follows in a few steps, which the development also derives by hand (comm-cx0-Q): since $Q \approx TcTc$ and Q is an involution, $c\,Q\,c \approx c\,(TcTc)\,c \approx cTcT \approx Q^{-1} \approx Q$, so c commutes with Q. The last orbit of the table is not found while only the first two are known, and is found at once after the others. □

E.8 The Relations between the Cycles

This part follows the generated modules Four.Part1–Four.Part14 and Four.Relations.

The forms of a cycle. With $T = \mathrm{CCX}_{30\to1}$ and $C = \mathrm{CCX}_{21\to0}$ as in Appendix E.3, the circuit K is equal modulo $\mathcal{T}$ to $T\,C\,T\,C$ as a word of placed gates, by two uses of the commutation of σ_0 and σ_2. Each Ω_i is K between a symmetry word and its reverse, so by Lemma E.8 it is equal to a word $T_iC_iT_iC_i$ of four placed Toffoli gates. Let wire a change from position i to $i+1$, wire b from $i+1$ to $i+2$, and let c and d be the other two wires. Then one of the two gates flips wire b where wire a is as in position $i+1$ and wire d as in position i, and the other flips wire a where wire b is as in position $i+1$ and wire c as in position i. The wires c and d may be given to the two gates either way round, so a cycle has two such words, its *forms*. For Ω_9 they are $(\mathrm{CCX}_{20\to1}\,\mathrm{CCX}_{31\to0})^2$ and $(\mathrm{CCX}_{30\to1}\,\mathrm{CCX}_{21\to0})^2$.

Lemma E.10 (the two forms). *The two forms of each cycle are equal modulo $\mathcal{T}$.*

Proof. For Ω_9, a search finds a macro derivation of 17 moves. A symmetry of the cube carries the positions 9, 10, 11 to i, $i+1$, $i+2$, and it carries the two forms of Ω_9 to the two forms of Ω_i, so the equation is transported as in Proposition E.9. □

Shapes. A symmetry of the cube carries three corners of a face to three corners of a face. The 105 instances of Lafont's relations fall into 24 orbits under the symmetries, their *shapes*: one for (order), two for (near), two for (far), and 19 for (apart).

Proposition E.11 (order$^\mathsf{T}$, near$^\mathsf{T}$, far$^\mathsf{T}$, apart$^\mathsf{T}$). *The* 105 *instances of* (order), (near), (far), *and* (apart) *between the circuits* Ω_i *hold modulo* $\mathcal{T}$.

Proof. *One relator for each shape.* For an instance $l = r$ of a shape, with each cycle in one of its forms, the *relator* is the word $l\,r^{-1}$ of placed gates, where r^{-1} is r reversed. It has 12 letters for (order), 20 for (near), 24 for (far), and 16 for (apart). For every shape but (order), a best-first search on the length of the word finds a macro derivation that reduces the relator to the empty word, in 22 to 123 moves. The forms matter to the search: for four shapes of (apart) the relator has a cycle in its second form, with which the search ends in seconds, while with both cycles in their first form it exceeds its memory bound.

Order. The search does not find $K^3 \approx \varepsilon$ directly. Let $T = \mathrm{CCX}_{20\to1}$ and $C = \mathrm{CCX}_{31\to0}$, so that $K \approx (T\,C)^2$, let $t = \mathrm{CX}_{0\to1}$, and let $z = C\,t\,C\,t\,C$, a word on the wires 0, 1, and 3. The search finds $T\,C\,T\,C\,T \approx t\,C\,T\,C\,t$ in 27 moves and $T\,z \approx z\,T$ in five. Then

$$K^3 \approx (TCTCT)\,C\,(TCTCT)\,C \approx t\,C\,T\,z\,T\,C\,t\,C \approx t\,C\,z\,T\,T\,C\,t\,C \approx t\,C\,z\,C\,t\,C,$$

and the last word is on three wires and has the function of the identity, so it is reduced by narrow moves. The composed derivation has 77 moves.

From a shape to its instances. Let a symmetry carry the cycles of the shape's instance to those of another instance. Conjugating the relator by a symmetry word gives, gate by gate and by Lemma E.8, a relator of the other instance in some forms of its cycles. Lemma E.10 changes these to the forms that the words Ω_i unfold to. Finally $l \approx l\,r^{-1}r \approx r$, where $r^{-1}r$ is inserted pair by pair. All of this is generated: the 24 relators, the lemma of the two forms for the fourteen cycles, and the 105 instances are equations of the engine between lists of generators. □

In Agda each family is a function of its indices with a Boolean guard, so that the indices out of range need no case.

E.9 The Gates as Products of Cycles

This part follows Four.Relations and the module Equivalence. For even $i \le 12$, the positions $i, \ldots, i+3$ are the four corners of a face of the cube. The two cycles on them generate the twelve even permutations of these corners, and two of these are Toffoli gates: $\Omega_{i+1}\Omega_i$ exchanges i with $i+1$ and $i+2$ with $i+3$, and $\Omega_{i+1}\Omega_i\Omega_i\Omega_{i+1}$ exchanges i with $i+3$ and $i+1$ with $i+2$. Write t_i and t'_i for the placed Toffoli gates with these functions:

$$\begin{array}{llll} t_0 = \mathrm{CCX}_{\bar3\bar2\to0}, & t_4 = \mathrm{CCX}_{\bar32\to0}, & t_8 = \mathrm{CCX}_{32\to0}, & t_{12} = \mathrm{CCX}_{3\bar2\to0}, \\ t_2 = \mathrm{CCX}_{\bar31\to0}, & t_6 = \mathrm{CCX}_{2\bar1\to0}, & t_{10} = \mathrm{CCX}_{31\to0}, & \\ t'_0 = \mathrm{CCX}_{\bar3\bar2\to1}, & t'_4 = \mathrm{CCX}_{\bar32\to1}, & t'_8 = \mathrm{CCX}_{32\to1}, & t'_{12} = \mathrm{CCX}_{3\bar2\to1}, \\ t'_2 = \mathrm{CCX}_{\bar31\to2}, & t'_6 = \mathrm{CCX}_{2\bar1\to3}, & t'_{10} = \mathrm{CCX}_{31\to2}. & \end{array}$$

Lemma E.12. *For even* $i \le 12$, $\Omega_{i+1}\Omega_i \approx t_i$ *and* $\Omega_i\Omega_{i+1} \approx t_i\,t'_i$, *hence* $t'_i \approx \Omega_{i+1}\Omega_i\Omega_i\Omega_{i+1}$. *Moreover* $\Omega_1\,t_2\,\Omega_4 \approx \mathrm{CCX}_{\bar30\to1}\,t_2\,\mathrm{CCX}_{\bar30\to1}$ *and* $\Omega_9\,t_{10}\,\Omega_{12} \approx \mathrm{CCX}_{30\to1}\,t_{10}\,\mathrm{CCX}_{30\to1}$, *and* $\mathrm{CCX}_{21\to0} \approx t_4\,t_6\,t_8$.

Proof. For each of the seventeen equations, a search finds a macro derivation of its relator, of 4 to 12 letters, in at most 27 moves. □

Lemma E.13 (gate-X$^\mathsf{T}$, gate-CX$^\mathsf{T}$, gate-CCX$^\mathsf{T}$, gate-Ex$_0{}^\mathsf{T}$, gate-Ex$_1{}^\mathsf{T}$, gate-Ex$_2{}^\mathsf{T}$). *Each of* X_0, $\mathrm{CX}_{1\to0}$, $\mathrm{CCX}_{21\to0}$, σ_0, σ_1, *and* σ_2 *is equal modulo* $\mathcal{T}$ *to the circuit* $g(v)$ *of some word* v *in the cycles.*

Proof. By Lemma E.12 each t_i and t'_i is such a circuit, and so is $\mathrm{CCX}_{21\to0}$. The other gates are products of these by one three-wire step each, in the frame of three of the four wires. On the wires 0, 2, 3, the gate X_0 is $t_0\,t_4\,t_8\,t_{12}$, the Toffoli gates onto wire 0 for the four values of the wires 3 and 2.

On the wires 0, 1, 3, the gate $\mathrm{CX}_{1\to0}$ is $t_2\,t_{10}$, and σ_0 is the product of the two controlled swaps of Lemma E.12, which exchange the wires 0 and 1 where wire 3 is 0 and where it is 1. On the wires 1, 2, 3, we have $\mathrm{CX}_{2\to1} = t'_4\,t'_8$ and $\mathrm{CX}_{1\to2} = t'_2\,t'_{10}$, so $\sigma_1 = \mathrm{CX}_{2\to1}\,\mathrm{CX}_{1\to2}\,\mathrm{CX}_{2\to1}$ is a product of six of the t'_i. Also $\mathrm{X}_1 = t'_0\,t'_4\,t'_8\,t'_{12}$, which gives the open controls in $\mathrm{CX}_{3\to2} = t'_{10}\,(\mathrm{X}_1\,t'_{10}\,\mathrm{X}_1)$ and $\mathrm{CX}_{2\to3} = (\mathrm{X}_1\,t'_6\,\mathrm{X}_1)\,t'_6$, so $\sigma_2 = \mathrm{CX}_{3\to2}\,\mathrm{CX}_{2\to3}\,\mathrm{CX}_{3\to2}$ is a product of thirty of the t'_i. For X_0, $\mathrm{CX}_{1\to0}$, $\mathrm{CCX}_{21\to0}$, and σ_0 the word v reached is f of the gate; for σ_1 and σ_2 it has 24 and 120 cycles, where f has 12 and 24. □

Lemma E.14 (to-f). *If a four-wire circuit c is equal modulo $\mathcal{T}$ to $g(v)$ for some word v in the cycles, then $c \approx g(f(c))$.*

Proof. By Proposition E.1 and Lemma E.3, v permutes the positions as c maps the states, and so does $f(c)$. So v and $f(c)$ are equal modulo Lafont's relations, by his completeness. By Proposition E.11 each instance of his relations, read through g, holds modulo $\mathcal{T}$, so g carries that derivation to $g(v) \approx g(f(c))$ (Lemmas.FromCycles). □

Proposition E.15 (lemma-g-f^{T}). *The six gate equations of the translated relation hold modulo $\mathcal{T}$, and every generator x on four wires satisfies $x \approx g(f(x))$.*

Proof. For the six gates on their own wires, combine Lemmas E.13 and E.14. A generator further up is the gate between swaps by the naturality of the swap, hence the circuit of some word in the cycles, and Lemma E.14 applies again. □

E.10 The Second Step and Completeness

This part follows the modules Equivalence and MainTheorem.

Proposition E.16 (s⇒t, t⇒s, rules≅relations). *Two four-wire circuits are equal modulo the translated relation if and only if they are equal modulo $\mathcal{T}$.*

Proof. The step is between two relations on the same circuits, with the identity translations, so its third and fourth facts are trivial and the other two say that each relation derives the equations of the other. The structural rules are the same on both sides. The equations of the translated relation hold modulo $\mathcal{T}$ by Propositions E.11 and E.15. Conversely, by Proposition E.1 the two sides of an equation that holds modulo $\mathcal{T}$ denote the same function, so they are equal modulo the translated relation, which is complete by Proposition E.4. □

Proof of Theorem 6.1. Soundness is Proposition E.1. For $k \le 3$, completeness and the group G_k are Proposition E.5. On four wires, two circuits with the same function are equal modulo the translated relation by Proposition E.4, hence modulo $\mathcal{T}$ by Proposition E.16 (complete^{R}). By the same two propositions, G_4 is the group of the circuits modulo the translated relation, which reading a circuit as the permutation that it carries out on the positions maps isomorphically onto Alt(16) (presentation^{R}). The development also proves the four facts of the first step for $\mathcal{T}$ itself, with the same f and g (Theorem-Rules-iso-Cycles): toward Lafont's words by soundness and his completeness, and toward the circuits by Propositions E.11 and E.15. □

E.11 The Canonical Form

This part follows the module NormalForm and the library's Alternating. The stairs $s_d(j)$ and Lafont's canonical forms among N points are those of Section 6: a canonical form is the product of one stairs for each $d \le N-3$, the highest first, with $d \le j_d \le N-1$, so there are $N(N-1)\cdots3 = N!/2$ canonical forms. We number the points from 0, where Lafont numbers them from 1, and we write products of permutations in the order of circuits, where he writes them in the order of composition.

The stairs to the last point repeats its last cycle; Lafont treats this case as a separate type of canonical form. The targets of an even permutation π are computed one stairs at a time: $\pi_0 = \pi$, and for each d in turn, $j_d = \pi_d(d)$ and $\pi_{d+1} = \pi_d\, s_d(j_d)^{-1}$, which fixes the points $0, \ldots, d$. At the end, π_{N-2} is an even permutation that fixes all points but the last two, so it is the identity, and π is the permutation of the canonical form with the targets $j_0, \ldots, j_{N-3}$. The library proves Lafont's Lemmas 6 and 8.

Lemma E.17 (Lafont; Surjectivity.canonical, UniqueNormalForm, Normalization). *Every even permutation of N points is the permutation of exactly one canonical form, and every word in the cycles is equal, modulo Lafont's relations, to a canonical form.*

Proof. For existence, the stairs from 0 to $\pi(0)$ is divided out of an even permutation π. What is left fixes the point 0 and is an even permutation of the other points, to which the construction applies again. This is the computation of the targets given above. For uniqueness, the first target of a canonical form is where its permutation carries 0, and cancelling the last stairs leaves two canonical forms on one point fewer. For the last claim, Lafont gives seven rewriting rules that follow from his relations and a lemma that carries a cycle through a stairs (his Lemma 7), and he leaves the induction to the reader: one cycle at a time is pushed through a canonical form, as in his proof for the symmetric groups. The library follows his proofs of existence and uniqueness and carries out this induction. □

Write $S_d(j)$ for the circuit $g(s_d(j))$ of a stairs. The canonical circuits of Section 6 are g of Lafont's canonical forms for $N = 16$, letter for letter (canon-g): $\mathrm{Can}(j_0, \ldots, j_{13}) = S_{13}(j_{13}) \cdots S_1(j_1)\, S_0(j_0)$. In Agda a tuple of targets is a tuple of stairs, nf w is the canonical form that Lemma E.17 gives for the word $f(w)$, and canon is g of a canonical form.

Proof of Theorem 6.2. *Existence* (normal-form^{R}, nf-sem). Let u be a four-wire circuit and v the canonical form equal to $f(u)$ modulo Lafont's relations. His relations are sound, so v has the permutation of $f(u)$, which by Lemma E.3 is the permutation that u carries out, and $g(v)$ carries out the same permutation. So u and the canonical circuit $g(v)$ have the same function, and they are equal modulo $\mathcal{T}$ by Theorem 6.1. By the uniqueness in Lemma E.17, v is the canonical form that the existence proof computes from that permutation, so its targets are those of the permutation.

Uniqueness and the bijection (canon-unique, canon-onto, nf-canon, nf-cong^{R}). Two canonical circuits with the same function are g of two canonical forms with the same permutation, by Lemma E.3, and these are the same by Lemma E.17. Every even permutation of the positions is the permutation of a canonical form, whose circuit carries it out. If u is equal modulo $\mathcal{T}$ to a canonical circuit, then by Proposition E.1 they carry out the same permutation, so that circuit is the one above. The count $16 \cdot 15 \cdots 3 = 16!/2$ is the number of tuples of targets. □

Example. The canonical circuit that a circuit is equal to is thus read off its permutation. The Toffoli gate $\mathrm{CCX}_{21\to0}$ exchanges the states 0110 and 0111, which are the positions 4 and 5, and the states 1111 and 1110, the positions 10 and 11, and it fixes the other twelve positions. Let π be this permutation. It fixes the positions below 4, so $j_d = d$ and $\pi_{d+1} = \pi$ for $d \le 3$. Then $j_4 = \pi(4) = 5$, and dividing by the stairs $s_4(5) = \omega_4$ leaves π_5, which exchanges 5 with 6 and still 10 with 11. In the same way $j_d = d + 1$ for $d = 5, \ldots, 8$, the first exchange moving up one position at each step, until π_9 is the cycle ω_9 itself, so that $j_9 = 10$, π_{10} is the identity, and $j_d = d$ for $d \ge 10$. The canonical form of π is therefore $\omega_9\, \omega_8\, \omega_7\, \omega_6\, \omega_5\, \omega_4$ (Figure 11(c)), and the gate is equal modulo $\mathcal{T}$ to the canonical circuit

$$S_9(10)\, S_8(9)\, S_7(8)\, S_6(7)\, S_5(6)\, S_4(5) = \Omega_9\, \Omega_8\, \Omega_7\, \Omega_6\, \Omega_5\, \Omega_4,$$

six stairs of one cycle each, the other eight being empty (Figure 12(b)). With every cycle written out as it is defined in Appendix E.3, this circuit has 104 generators, twelve of them in Ω_9. Other

circuits have longer stairs. The swap σ_1, for example, is equal to the canonical circuit

$$S_{10}(12)\,S_8(12)\,S_4(6)\,S_2(6) = (\Omega_{10}\,\Omega_{11})\,(\Omega_8\,\Omega_9\,\Omega_{10}\,\Omega_{11})\,(\Omega_4\,\Omega_5)\,(\Omega_2\,\Omega_3\,\Omega_4\,\Omega_5).$$

Lafont's rewriting also computes the canonical form, but on the word of a circuit it passes through millions of letters, and the development never runs it.

E.12 Implementation

The development consists of 38 modules in `Examples/Groups/Alt16-RevCir`, all reached from MainTheorem, and every module is checked with Agda's `--safe` flag. Given the interfaces of the shared library, Agda checks them in about 45 seconds. Of its 28,000 lines, 4,300 are written by hand, and the rest are generated derivations: 2,900 lines on two and three wires, and 21,000 lines on four, in 788 equations with 15,607 steps. In a generated module a list of generators is a string with one character for each generator, and the Boolean that decides a step is a hidden argument that Agda fills in once it has evaluated to true.

The scripts that find the derivations are part of the development. They are the Knuth–Bendix completion on two and three wires, the search for the pair lemmas of Proposition E.9, which takes 20 seconds, the best-first search for the relators of Proposition E.11 and Lemma E.12, which takes at most a minute for each, and a compiler from macro derivations to steps of the engine, which replays every step. Each search is bounded, and runs within 1.5 GB. Two changes brought the last four relators within this bound: another form of one cycle for three of them, and a compact storage of the words that the search has seen for the fourth.

The development is translated to Lean module for module, with the same generated derivations: its 38 modules, and sixteen more for what it imports from the library of EquaCert, Lafont's presentation among it (Appendix C).